

About Project



Problem Statement

Recruiting and retaining drivers is seen by industry watchers as a tough battle for Ola. Churn among drivers is high and it's very easy for drivers to stop working for the service on the fly or jump to competitor depending on the rates.

As the companies get bigger, the high churn could become a bigger problem. To find new drivers, Ola is casting a wide net, including people who don't have cars for jobs. But this acquisition is really costly. Losing drivers frequently impacts the morale of the organization and acquiring new drivers is more expensive than retaining existing ones.

Main Task

You are working as a data scientist with the Analytics Department of Ola, focused on driver team attrition. You are provided with the monthly information for a segment of drivers for 2019 and 2020 and tasked to predict whether a driver will be leaving the company or not based on their attributes like

Demographics (city, age, gender etc.) Tenure information (joining date, Last Date) Historical data regarding the performance of the driver (Quarterly rating, Monthly business acquired, grade, Income)

Driver Dataset Documentation

Fields:

1. **MMMM-YY : Reporting Date (Monthly)**

- Represents the monthly reporting period in `MMMM-YY` format.

2. **Driver_ID : Unique ID for drivers**

- A unique identifier assigned to each driver.

3. **Age : Age of the driver**

- Age of the driver in years.

4. **Gender : Gender of the driver**

- Encoded values:
 - `0` : Male
 - `1` : Female

5. **City : City Code of the driver**

- Code representing the city where the driver is based.

6. **Education_Level : Education level**

- Encoded values:
 - `0` : 10+
 - `1` : 12+
 - `2` : Graduate

7. **Income : Monthly average income of the driver**

- Represents the driver's average monthly income.

8. **Date Of Joining : Joining date for the driver**

- The date when the driver joined.

9. **LastWorkingDate : Last date of working for the driver**

- The last working date recorded for the driver.

10. **Joining Designation : Designation of the driver at the time of joining**

- The job title/designation when the driver started.

11. **Grade : Grade of the driver at the time of reporting**

- The grade level assigned to the driver.

12. **Total Business Value : Total business value acquired by the driver in a month**

- The business value generated by the driver in a given month:
 - Positive value indicates revenue.
 - Negative value indicates cancellations, refunds, or car EMI adjustments.

13. **Quarterly Rating : Quarterly rating of the driver**

- Ratings range from 1 to 5 (higher is better).

Concepts Tested

- Ensemble Learning- Bagging
- Ensemble Learning- Boosting
- KNN Imputation of Missing Values
- Working with an imbalanced dataset

Data Analysis and Machine Learning Workflow

Steps Overview

1. Data Import and Initial Analysis

- Import the dataset and perform exploratory analysis steps such as:
 - Checking the structure and characteristics of the dataset.
 - Understanding distributions and summary statistics.

2. Data Preparation

- Convert date-like features to their respective data types.

- Identify and handle missing values.
 - Prepare data for **KNN Imputation** by considering only numerical features.
- Aggregate data to remove multiple occurrences of the same driver's data:
 - Start with an empty DataFrame containing unique `Driver_IDs` .
 - Use a **GroupBy Driver_ID** approach to bring all features to the same level (similar to Delhivery business case).

3. Feature Engineering

- **Create New Columns:**
 1. **Quarterly Rating Increase:**
 - A column indicating whether the quarterly rating has increased for a driver (`1` for increased, `0` otherwise).
 2. **Target Variable:**
 - A column (`target`) indicating whether the driver has left the company (`1` if `LastWorkingDate` is present, `0` otherwise).
 3. **Monthly Income Increase:**
 - A column indicating whether the driver's monthly income has increased (`1` for increased, `0` otherwise).

4. Statistical Summary

- Generate a statistical summary of the derived dataset.
- Check the correlation among independent variables and analyze their interactions.

5. Data Transformation

- Apply **One-Hot Encoding** for categorical variables.
- Address **Class Imbalance** using appropriate techniques.
- Standardize training data to ensure uniformity in feature scaling.

6. Model Building

- Use **Ensemble Learning Methods** such as:
 - **Bagging** (e.g., Random Forest, Extra Trees).
 - **Boosting** (e.g., Gradient Boosting, XGBoost, LightGBM).
- Perform **Hyperparameter Tuning** to optimize model performance.

7. Results Evaluation

- Generate and evaluate performance met

Problem Statement and Solution Workflow

1. Define Problem Statement and Perform Exploratory Data Analysis (10 Points)

Problem Definition

The goal is to analyze the provided driver dataset to derive insights, predict driver attrition, and understand factors influencing driver performance, income growth, and attrition. Additional views include understanding trends in driver ratings, income, and business contribution for decision-making.

Exploratory Data Analysis

Observations on Data

1. **Shape of Data:** Note the number of rows and columns.
2. **Data Types:** Identify and verify data types for all attributes. Convert categorical attributes to `category` data type if necessary.
3. **Missing Value Detection:** Identify attributes with missing values for imputation.
4. **Statistical Summary:** Calculate mean, median, standard deviation, and percentiles for numerical attributes.

Univariate Analysis

- **Continuous Variables:** Plot histograms or density plots to examine distributions (e.g., income, age, total business value).
- **Categorical Variables:** Use barplots or countplots for attributes like `Gender`, `Education_Level`, and `Grade`.

Bivariate Analysis

- Examine relationships between variables such as:
 - Age vs Income (scatterplot).
 - Quarterly Rating vs Total Business Value (boxplot or line chart).
 - Education_Level vs Income (grouped barplot).

Insights from EDA

1. Comment on the range of attributes (e.g., age ranges, income brackets).
2. Identify outliers using boxplots.
3. Discuss distributions (e.g., skewness, normality).
4. Highlight key relationships (e.g., how education level influences income).

Comments for Each Plot

- Univariate: Discuss skewness, peaks, and variability.
 - Bivariate: Discuss strength, direction, and significance of relationships.
-

2. Data Preprocessing (50 Points)

Steps:

1. KNN Imputation

- Handle missing values using K-Nearest Neighbors based on numerical features.

2. Feature Engineering

- Create derived attributes:
 - **Quarterly Rating Increase:** Binary feature indicating a rating increase.
 - **Target:** Binary feature indicating driver attrition.
 - **Monthly Income Increase:** Binary feature indicating income growth.

3. Class Imbalance Treatment

- Apply SMOTE, undersampling, or oversampling to balance the target variable.

4. Standardization

- Standardize numerical features to ensure consistent scaling.

5. Encoding

- Apply one-hot encoding for categorical variables to make them model-compatible.
-

3. Model Building (20 Points)

Algorithms:

1. Bagging Algorithm

- Example: Random Forest to leverage bootstrapped datasets for robust performance.

2. Boosting Algorithm

- Example: Gradient Boosting (XGBoost, LightGBM) to focus on harder-to-predict samples.

Hyperparameter Tuning

- Use GridSearchCV or RandomizedSearchCV to optimize model parameters.
-

4. Results Evaluation (10 Points)

1. ROC AUC Curve

- Plot the curve and comment on the model's ability to distinguish between classes.

2. Classification Report

- Generate a confusion matrix, along with precision, recall, F1-score, and accuracy metrics.
-

5. Actionable Insights & Recommendations (10 Points)

Insights

- Provide key findings on driver behavior, performance, and attrition.
- Highlight drivers' income growth and its correlation with other variables.

Recommendations

- Propose actionable steps for reducing attrition, improving driver satisfaction, and optimizing operations.

- Suggest strategies for targeted interventions based on insights (e.g., reward systems for high performers).

Solution

```
In [ ]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

import warnings as war
war.filterwarnings('ignore')
```

```
In [ ]: !gdown 1eNcd23917r1QlTkl95pf3AF30IEngw0d
```

Downloading...

From: <https://drive.google.com/uc?id=1eNcd23917r1QlTkl95pf3AF30IEngw0d> (<https://drive.google.com/uc?id=1eNcd23917r1QlTkl95pf3AF30IEngw0d>)

To: /content/10_ola_driver_scaler.csv

100% 1.13M/1.13M [00:00<00:00, 166MB/s]

```
In [ ]: df = pd.read_csv('10_ola_driver_scaler.csv')
df.head()
```

Out[3]:

	Unnamed: 0	MMM-YY	Driver_ID	Age	Gender	City	Education_Level	Income	Dateofjoining	LastWorkingDate	Joining Designation	Grade	Total Business Value	Qual F
0	0	01/01/19	1	28.0	0.0	C23	2	57387	24/12/18	NaN	1	1	2381060	
1	1	02/01/19	1	28.0	0.0	C23	2	57387	24/12/18	NaN	1	1	-665480	
2	2	03/01/19	1	28.0	0.0	C23	2	57387	24/12/18	03/11/19	1	1	0	
3	3	11/01/20	2	31.0	0.0	C7	2	67016	11/06/20	NaN	2	2	0	
4	4	12/01/20	2	31.0	0.0	C7	2	67016	11/06/20	NaN	2	2	0	

Data Profiling

- MMMM-YY : Reporting Date (Monthly)
- Driver_ID : Unique id for drivers
- Age : Age of the driver
- Gender : Gender of the driver – Male : 0, Female: 1
- City : City Code of the driver
- Education_Level : Education level – 0 for 10+ ,1 for 12+ ,2 for graduate
- Income : Monthly average Income of the driver
- Date Of Joining : Joining date for the driver
- LastWorkingDate : Last date of working for the driver
- Joining Designation : Designation of the driver at the time of joining
- Grade : Grade of the driver at the time of reporting
- Total Business Value : The total business value acquired by the driver in a month (negative business indicates cancellation/refund or car EMI adjustments)
- Quarterly Rating : Quarterly rating of the driver: 1,2,3,4,5 (higher is better)

Basic checks

```
In [ ]: df.columns
```

```
Out[4]: Index(['Unnamed: 0', 'MMM-YY', 'Driver_ID', 'Age', 'Gender', 'City',  
             'Education_Level', 'Income', 'Dateofjoining', 'LastWorkingDate',  
             'Joining Designation', 'Grade', 'Total Business Value',  
             'Quarterly Rating'],  
            dtype='object')
```

```
###
```

```
In [ ]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 19104 entries, 0 to 19103
Data columns (total 14 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Unnamed: 0             19104 non-null  int64
1   MMM-YY                 19104 non-null  object
2   Driver_ID              19104 non-null  int64
3   Age                    19043 non-null  float64
4   Gender                 19052 non-null  float64
5   City                   19104 non-null  object
6   Education_Level        19104 non-null  int64
7   Income                  19104 non-null  int64
8   Dateofjoining           19104 non-null  object
9   LastWorkingDate         1616 non-null   object
10  Joining Designation     19104 non-null  int64
11  Grade                   19104 non-null  int64
12  Total Business Value    19104 non-null  int64
13  Quarterly Rating        19104 non-null  int64
dtypes: float64(2), int64(8), object(4)
memory usage: 2.0+ MB
```

- Reporting data is object type instead of date
- Date of Joining and last working date is also object instead of date type

```
In [ ]: df.isnull().sum()
```

Out [6]:

	0
Unnamed: 0	0
MMM-YY	0
Driver_ID	0
Age	61
Gender	52
City	0
Education_Level	0
Income	0
Dateofjoining	0
LastWorkingDate	17488
Joining Designation	0
Grade	0
Total Business Value	0
Quarterly Rating	0

dtype: int64

```
In [ ]: df.describe()
```

Out [7]:

	Unnamed: 0	Driver_ID	Age	Gender	Education_Level	Income	Joining Designation	Grade	Total Business Value	Qu
count	19104.000000	19104.000000	19043.000000	19052.000000	19104.000000	19104.000000	19104.000000	19104.000000	1.910400e+04	19104.0
mean	9551.500000	1415.591133	34.668435	0.418749	1.021671	65652.025126	1.690536	2.252670	5.716621e+05	2.0
std	5514.994107	810.705321	6.257912	0.493367	0.800167	30914.515344	0.836984	1.026512	1.128312e+06	1.0
min	0.000000	1.000000	21.000000	0.000000	0.000000	10747.000000	1.000000	1.000000	-6.000000e+06	1.0
25%	4775.750000	710.000000	30.000000	0.000000	0.000000	42383.000000	1.000000	1.000000	0.000000e+00	1.0
50%	9551.500000	1417.000000	34.000000	0.000000	1.000000	60087.000000	1.000000	2.000000	2.500000e+05	2.0
75%	14327.250000	2137.000000	39.000000	1.000000	2.000000	83969.000000	2.000000	3.000000	6.997000e+05	3.0
max	19103.000000	2788.000000	58.000000	1.000000	2.000000	188418.000000	5.000000	5.000000	3.374772e+07	4.0

```
In [ ]: df.describe(include = 'object')
```

Out [8]:

	MMM-YY	City	Dateofjoining	LastWorkingDate
count	19104	19104	19104	1616
unique	24	29	869	493
top	01/01/19	C20	23/07/15	29/07/20
freq	1022	1008	192	70

```
In [ ]: df.shape[0]
```

Out [9]: 19104

```
In [ ]: df['Driver_ID'].nunique()
```

```
Out[10]: 2381
```

So for each driver there are multiple rows, let check them below

```
In [ ]: df['Driver_ID'].value_counts()
```

```
Out[11]:
```

	count
Driver_ID	
26	24
25	24
2784	24
2740	24
1077	24
...	...
2023	1
2030	1
1027	1
1029	1
1030	1

2381 rows × 1 columns

dtype: int64

```
In [ ]: #Dropping tge Unnamed column, since it is not useful further  
df.drop(columns = ['Unnamed: 0'], axis = 1, inplace = True)
```

```
In [ ]: df[df['Driver_ID']== 2110]
```

Out[13]:

	MMM-YY	Driver_ID	Age	Gender	City	Education_Level	Income	Dateofjoining	LastWorkingDate	Joining Designation	Grade	Total Business Value	Quarterly Rating
14143	01/01/19	2110	36.0	0.0	C19	0	131847	21/01/16	NaN	5	5	0	1
14144	02/01/19	2110	36.0	0.0	C19	0	131847	21/01/16	NaN	5	5	0	1
14145	03/01/19	2110	37.0	0.0	C19	0	131847	21/01/16	NaN	5	5	0	1
14146	04/01/19	2110	37.0	0.0	C19	0	131847	21/01/16	NaN	5	5	0	1
14147	05/01/19	2110	37.0	0.0	C19	0	131847	21/01/16	NaN	5	5	0	1
14148	06/01/19	2110	37.0	0.0	C19	0	131847	21/01/16	NaN	5	5	0	1
14149	07/01/19	2110	37.0	0.0	C19	0	131847	21/01/16	NaN	5	5	0	1
14150	08/01/19	2110	37.0	0.0	C19	0	131847	21/01/16	NaN	5	5	0	1
14151	09/01/19	2110	37.0	0.0	C19	0	131847	21/01/16	NaN	5	5	0	1
14152	10/01/19	2110	37.0	0.0	C19	0	131847	21/01/16	NaN	5	5	0	1
14153	11/01/19	2110	37.0	0.0	C19	0	131847	21/01/16	NaN	5	5	923600	1
14154	12/01/19	2110	37.0	0.0	C19	0	131847	21/01/16	NaN	5	5	407000	1
14155	01/01/20	2110	37.0	0.0	C19	0	131847	21/01/16	NaN	5	5	1060560	2
14156	02/01/20	2110	37.0	0.0	C19	0	131847	21/01/16	NaN	5	5	1779250	2
14157	03/01/20	2110	38.0	0.0	C19	0	131847	21/01/16	NaN	5	5	8947330	2
14158	04/01/20	2110	38.0	0.0	C19	0	131847	21/01/16	NaN	5	5	1479180	2
14159	05/01/20	2110	38.0	0.0	C19	0	131847	21/01/16	NaN	5	5	400730	2
14160	06/01/20	2110	38.0	0.0	C19	0	131847	21/01/16	NaN	5	5	500000	2
14161	07/01/20	2110	38.0	0.0	C19	0	131847	21/01/16	NaN	5	5	451340	2
14162	08/01/20	2110	38.0	0.0	C19	0	131847	21/01/16	NaN	5	5	800810	2
14163	09/01/20	2110	38.0	0.0	C19	0	131847	21/01/16	NaN	5	5	1208970	2
14164	10/01/20	2110	38.0	0.0	C19	0	131847	21/01/16	NaN	5	5	948000	1
14165	11/01/20	2110	38.0	0.0	C19	0	131847	21/01/16	NaN	5	5	560730	1
14166	12/01/20	2110	38.0	0.0	C19	0	131847	21/01/16	NaN	5	5	719650	1



From above we can see that there are 24 rows for the same driver, each line for each reporting month from Jan-2019 to Dec-2020. No of row in a way represents no of months driver is with the company.

Changing column data types

```
In [ ]: df["MMM-YY"] = pd.to_datetime(df["MMM-YY"])
df["Dateofjoining"] = pd.to_datetime(df["Dateofjoining"])
df["LastWorkingDate"] = pd.to_datetime(df["LastWorkingDate"])
```

```
In [ ]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 19104 entries, 0 to 19103
Data columns (total 13 columns):
#   Column                Non-Null Count  Dtype
---  -
0   MMM-YY                19104 non-null  datetime64[ns]
1   Driver_ID             19104 non-null  int64
2   Age                   19043 non-null  float64
3   Gender                19052 non-null  float64
4   City                  19104 non-null  object
5   Education_Level       19104 non-null  int64
6   Income                19104 non-null  int64
7   Dateofjoining         19104 non-null  datetime64[ns]
8   LastWorkingDate       1616 non-null   datetime64[ns]
9   Joining Designation   19104 non-null  int64
10  Grade                 19104 non-null  int64
11  Total Business Value  19104 non-null  int64
12  Quarterly Rating      19104 non-null  int64
dtypes: datetime64[ns](3), float64(2), int64(7), object(1)
memory usage: 1.9+ MB
```


KNN Imputation

```
In [ ]: # KNN imputation will only work on the numeric columns for missing value treatment
df_num_cols = df.select_dtypes(['int', 'float']) # data frame with only numeric columns
df_num_cols.head()
```

Out[16]:

	Driver_ID	Age	Gender	Education_Level	Income	Joining Designation	Grade	Total Business Value	Quarterly Rating
0	1	28.0	0.0	2	57387	1	1	2381060	2
1	1	28.0	0.0	2	57387	1	1	-665480	2
2	1	28.0	0.0	2	57387	1	1	0	2
3	2	31.0	0.0	2	67016	2	2	0	1
4	2	31.0	0.0	2	67016	2	2	0	1

==

==

```
In [ ]: from sklearn.impute import KNNImputer

# Initializing the imputer
Imputer = KNNImputer(n_neighbors= 5, weights = 'uniform', metric = 'nan_euclidean')
# Fitting the imputer on the data
df_new = Imputer.fit_transform(df_num_cols)
df_new = pd.DataFrame(df_new, columns = df_num_cols.columns)
df_new.isnull().sum()
```

```
Out[17]:
```

	0
Driver_ID	0
Age	0
Gender	0
Education_Level	0
Income	0
Joining Designation	0
Grade	0
Total Business Value	0
Quarterly Rating	0

dtype: int64

df_new only consist the numeric columns, we need to merge this with data with categorical & datetime columns

```
In [ ]: #getting all other columns
# passing both df and df_num_cols dataframe below to get the remaining columns
res = list(set(df).difference(set(df_num_cols)))
res
```

```
Out[18]: ['MMM-YY', 'City', 'LastWorkingDate', 'Dateofjoining']
```

```
In [ ]: df_other_cols = df[res] # data frame without the nueric columns
df_other_cols.head()
```

```
Out[19]:
```

	MMM-YY	City	LastWorkingDate	Dateofjoining
0	2019-01-01	C23	NaT	2018-12-24
1	2019-02-01	C23	NaT	2018-12-24
2	2019-03-01	C23	2019-03-11	2018-12-24
3	2020-11-01	C7	NaT	2020-11-06
4	2020-12-01	C7	NaT	2020-11-06

```
==
```

```
==
```

```
In [ ]: # creating new df after KNN imputation
df_new = pd.concat([df_new, df_other_cols], axis = 1)
df_new.head()
```

```
Out[20]:
```

	Driver_ID	Age	Gender	Education_Level	Income	Joining Designation	Grade	Total Business Value	Quarterly Rating	MMM-YY	City	LastWorkingDate	Dateofjoining
0	1.0	28.0	0.0	2.0	57387.0	1.0	1.0	2381060.0	2.0	2019-01-01	C23	NaT	2018-12-24
1	1.0	28.0	0.0	2.0	57387.0	1.0	1.0	-665480.0	2.0	2019-02-01	C23	NaT	2018-12-24
2	1.0	28.0	0.0	2.0	57387.0	1.0	1.0	0.0	2.0	2019-03-01	C23	2019-03-11	2018-12-24
3	2.0	31.0	0.0	2.0	67016.0	2.0	2.0	0.0	1.0	2020-11-01	C7	NaT	2020-11-06
4	2.0	31.0	0.0	2.0	67016.0	2.0	2.0	0.0	1.0	2020-12-01	C7	NaT	2020-11-06

```
==
```

```
==
```

```
In [ ]: df_new.isnull().sum()
```

```
Out[21]:
```

	0
Driver_ID	0
Age	0
Gender	0
Education_Level	0
Income	0
Joining Designation	0
Grade	0
Total Business Value	0
Quarterly Rating	0
MMM-YY	0
City	0
LastWorkingDate	17488
Dateofjoining	0

dtype: int64

There are still Null values in LastWorkingDate, missing value here means person had not left

```
In [ ]: df_new.head()
```

Out[22]:

	Driver_ID	Age	Gender	Education_Level	Income	Joining Designation	Grade	Total Business Value	Quarterly Rating	MMM- YY	City	LastWorkingDate	Dateofjoining
0	1.0	28.0	0.0	2.0	57387.0	1.0	1.0	2381060.0	2.0	2019-01-01	C23	NaT	2018-12-24
1	1.0	28.0	0.0	2.0	57387.0	1.0	1.0	-665480.0	2.0	2019-02-01	C23	NaT	2018-12-24
2	1.0	28.0	0.0	2.0	57387.0	1.0	1.0	0.0	2.0	2019-03-01	C23	2019-03-11	2018-12-24
3	2.0	31.0	0.0	2.0	67016.0	2.0	2.0	0.0	1.0	2020-11-01	C7	NaT	2020-11-06
4	2.0	31.0	0.0	2.0	67016.0	2.0	2.0	0.0	1.0	2020-12-01	C7	NaT	2020-11-06

Aggregating by Driver ID

```
In [ ]: import pandas as pd
import numpy as np

# Assuming 'df' is your DataFrame
df_mod = df_new.groupby('Driver_ID').aggregate({
    'Age': 'max', # Maximum age
    'Gender': 'last', # Unique gender values
    'Education_Level': 'max', # Highest education level
    'Income': 'mean', # Average income
    'Joining Designation': 'last', #last joining designation
    'Grade': 'first', #last grade
    'Total Business Value': 'sum', # Total business value
    'Quarterly Rating': 'last', # Average quarterly rating
    'LastWorkingDate': 'last', # Most recent working date
    'MMM-YY': 'size', # Count of records (number of months)
    'City': 'last', # Unique cities
    'Dateofjoining': lambda x: x.unique()[0] if len(x.unique()) == 1 else x.unique().tolist(), # Unique join
}).reset_index()
df_mod.head()
```

Out[23]:

	Driver_ID	Age	Gender	Education_Level	Income	Joining Designation	Grade	Total Business Value	Quarterly Rating	LastWorkingDate	MMM-YY	City	Dateofjoining
0	1.0	28.0	0.0	2.0	57387.0	1.0	1.0	1715580.0	2.0	2019-03-11	3	C23	2018-12-24
1	2.0	31.0	0.0	2.0	67016.0	2.0	2.0	0.0	1.0	NaT	2	C7	2020-11-06
2	4.0	43.0	0.0	2.0	65603.0	2.0	2.0	350000.0	1.0	2020-04-27	5	C13	2019-12-07
3	5.0	29.0	0.0	0.0	46368.0	1.0	1.0	120360.0	1.0	2019-03-07	3	C9	2019-01-09
4	6.0	31.0	1.0	1.0	78728.0	3.0	3.0	1265000.0	2.0	NaT	5	C11	2020-07-31

==

==

```
In [ ]: df_mod['Gender'].value_counts()
```

```
Out[24]:
```

	count
Gender	
0.0	1400
1.0	974
0.6	5
0.2	1
0.4	1

dtype: int64

```
In [ ]: # renaming the columns in df
df_mod = df_mod.rename(columns = {'MMM-YY': 'Months',
                                   'Education_Level': 'Education',
                                   'Joining Designation': 'Joining_Designation',
                                   'Total Business Value' : 'Total_Business_Value',
                                   'Quarterly Rating' : 'Last_Quarterly_Rating'
                                   })

df_mod.head()
```

```
Out[25]:
```

	Driver_ID	Age	Gender	Education	Income	Joining_Designation	Grade	Total_Business_Value	Last_Quarterly_Rating	LastWorkingDate	Months
0	1.0	28.0	0.0	2.0	57387.0	1.0	1.0	1715580.0	2.0	2019-03-11	3
1	2.0	31.0	0.0	2.0	67016.0	2.0	2.0	0.0	1.0	NaT	2
2	4.0	43.0	0.0	2.0	65603.0	2.0	2.0	350000.0	1.0	2020-04-27	5
3	5.0	29.0	0.0	0.0	46368.0	1.0	1.0	120360.0	1.0	2019-03-07	3
4	6.0	31.0	1.0	1.0	78728.0	3.0	3.0	1265000.0	2.0	NaT	5

Faeture Engineering

Quaterly Rating Increase

whether the quarterly rating has increased for that driver -for those whose quarterly rating has increased we assign the value 1

```
In [ ]: df.columns
```

```
Out[26]: Index(['MMM-YY', 'Driver_ID', 'Age', 'Gender', 'City', 'Education_Level',  
              'Income', 'Dateofjoining', 'LastWorkingDate', 'Joining Designation',  
              'Grade', 'Total Business Value', 'Quarterly Rating'],  
             dtype='object')
```



```
In [ ]: def quarter_rating_increase(rating):
    if len(rating) >= 2: #if the lines for that driver is greater or equal to 2
        return int(rating.iloc[-1] > rating.iloc[-2]) # comparing the last rating for that driver with previous c
    else:
        return 0

Quarterly_Rating_increased = df.groupby('Driver_ID')['Quarterly Rating'].apply(quarter_rating_increase).reset_index()

'''
After the apply() function, the result is a pandas Series with:

Index: The Driver_ID (from the groupby operation).
Values: The results of the qrt_rat_inc function (0 or 1 for each Driver_ID).
When you use .reset_index(), it converts this Series back into a DataFrame:

The original index (Driver_ID) becomes a column.
The resulting values (0 or 1) also become a column.
By specifying name='Quarterly_Rating_increased', you are naming this new column for the function results.
'''

df_final = pd.merge(left = df_mod,
                    right = Quarterly_Rating_increased,
                    on = 'Driver_ID',
                    how = 'outer' #to include all the rows from the both dataframes
                    )

df_final['Quarterly_Rating_increased'].value_counts(normalize = True)*100
```

Out[27]:

	proportion
Quarterly_Rating_increased	
0	99.328013
1	0.671987

dtype: float64

- less than 1% of the drivers have shown the increase in quaterly ratings.

```
In [ ]: df_final.head()
```

```
Out[28]:
```

	Driver_ID	Age	Gender	Education	Income	Joining_Designation	Grade	Total_Business_Value	Last_Quarterly_Rating	LastWorkingDate	Months
0	1.0	28.0	0.0	2.0	57387.0	1.0	1.0	1715580.0	2.0	2019-03-11	3
1	2.0	31.0	0.0	2.0	67016.0	2.0	2.0	0.0	1.0	NaT	2
2	4.0	43.0	0.0	2.0	65603.0	2.0	2.0	350000.0	1.0	2020-04-27	5
3	5.0	29.0	0.0	0.0	46368.0	1.0	1.0	120360.0	1.0	2019-03-07	3
4	6.0	31.0	1.0	1.0	78728.0	3.0	3.0	1265000.0	2.0	NaT	5

==

==

Target

Binary feature indicating driver attrition.

```
In [ ]: # Replace NaN values with 0 in LastWorkingDate column
df_final['LastWorkingDate'].fillna(0, inplace=True)

# Convert LastWorkingDate to numeric type to ensure correct comparison
df_final['LastWorkingDate'] = pd.to_numeric(df_final['LastWorkingDate'], errors='coerce')

# Define the churn function
def churn(data):
    if data == 0:
        return 0
    else:
        return 1 # 1 means the driver left the company

# Apply the churn function
df_final['Churn'] = df_final['LastWorkingDate'].apply(churn)

# Check the distribution of churn values
print(df_final['Churn'].value_counts())
```

```
Churn
1    1616
0     765
Name: count, dtype: int64
```

So basically 765 such driver who didn't left the company.

Monthly Income Increase

Binary feature indicating income growth.

```

In [ ]: # function to check whether monthly income increased or not
def income_increase(income):
    if len(income) >=2:
        return int(income.iloc[-1] > income.iloc[-2]) # checking whether latest income is higher than the previous
    else:
        0

# Fill missing values in Income column with 0 if needed
df['Income'] = df['Income'].fillna(0) # Fill NaN values in Income

# using the original df to compare the increase in the income level
Increased_Income = df.groupby('Driver_ID')['Income'].apply(income_increase).reset_index(name = 'Increased_Income')

# merging this dataframe with the df_final
df_final = pd.merge(left = df_final,
                    right = Increased_Income,
                    on = 'Driver_ID',
                    how = 'outer')

df_final['Increased_Income'].value_counts(normalize = True)*100

```

Out[30]:

	proportion	
Increased_Income		
0.0	99.5	
1.0	0.5	

dtype: float64

- only 0.5 % of driver shown the increase in the income levels. That is concerning.

```
In [ ]: df_final.head()
```

```
Out[31]:
```

	Driver_ID	Age	Gender	Education	Income	Joining_Designation	Grade	Total_Business_Value	Last_Quarterly_Rating	LastWorkingDate	Months
0	1.0	28.0	0.0	2.0	57387.0	1.0	1.0	1715580.0	2.0	NaN	3
1	2.0	31.0	0.0	2.0	67016.0	2.0	2.0	0.0	1.0	0.0	2
2	4.0	43.0	0.0	2.0	65603.0	2.0	2.0	350000.0	1.0	NaN	5
3	5.0	29.0	0.0	0.0	46368.0	1.0	1.0	120360.0	1.0	NaN	3
4	6.0	31.0	1.0	1.0	78728.0	3.0	3.0	1265000.0	2.0	0.0	5

```
==
```

```
==
```

```
In [ ]: df_final['Increased_Income'] = df_final['Increased_Income'].fillna(0)
```

```
# Check if the column is filled correctly
```

```
print(df_final['Increased_Income'].value_counts(normalize=True) * 100)
```

```
Increased_Income
```

```
0.0    99.538009
```

```
1.0     0.461991
```

```
Name: proportion, dtype: float64
```

```
In [ ]: df_final.isnull().sum()
```

Out[33]:

	0
--	---

Driver_ID	0
Age	0
Gender	0
Education	0
Income	0
Joining_Designation	0
Grade	0
Total_Business_Value	0
Last_Quarterly_Rating	0
LastWorkingDate	1616
Months	0
City	0
Dateofjoining	0
Quarterly_Rating_increased	0
Churn	0
Increased_Income	0

dtype: int64

Statistical Summary

```
In [ ]: df_final['Churn'].value_counts(normalize = True)*100
```

```
Out[34]:
```

	proportion
Churn	
1	67.870643
0	32.129357

dtype: float64

This shows that 67% drivers left the company, which is very high.

```
In [ ]: df_final.describe().T
```

```
Out[35]:
```

	count	mean	min	25%	50%	75%	max	std
Driver_ID	2381.0	1397.559009	1.0	695.0	1400.0	2100.0	2788.0	806.161628
Age	2381.0	33.770265	21.0	30.0	33.0	37.0	58.0	5.932235
Gender	2381.0	0.410584	0.0	0.0	0.0	1.0	1.0	0.491359
Education	2381.0	1.00756	0.0	0.0	1.0	2.0	2.0	0.81629
Income	2381.0	59232.460484	10747.0	39104.0	55285.0	75835.0	188418.0	28298.214012
Joining_Designation	2381.0	1.820244	1.0	1.0	2.0	2.0	5.0	0.841433
Grade	2381.0	2.078538	1.0	1.0	2.0	3.0	5.0	0.931321
Total_Business_Value	2381.0	4586741.822764	-1385530.0	0.0	817680.0	4173650.0	95331060.0	9127115.313446
Last_Quarterly_Rating	2381.0	1.427971	1.0	1.0	1.0	2.0	4.0	0.809839
LastWorkingDate	765.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
Months	2381.0	8.02352	1.0	3.0	5.0	10.0	24.0	6.78359
Dateofjoining	2381	2019-02-08 07:14:50.550189056	2013-04-01 00:00:00	2018-06-29 00:00:00	2019-07-21 00:00:00	2020-05-02 00:00:00	2020-12-28 00:00:00	NaN
Quarterly_Rating_increased	2381.0	0.00672	0.0	0.0	0.0	0.0	1.0	0.081716
Churn	2381.0	0.678706	0.0	0.0	1.0	1.0	1.0	0.467071
Increased_Income	2381.0	0.00462	0.0	0.0	0.0	0.0	1.0	0.067827



- The 'Driver_ID' column shows an unique count 2381.
- The average age of drivers is around 33 years, with a minimum age of 21 years and a maximum age of 58 years.
- The average income of drivers is approximately 59,232, with a wide range from 10,747 to 188,418.
- The 'Total_Business_Value' column has a significant range, with values ranging from negative to extremely high positive values, suggesting variations in business performance among drivers.
- Avg months is 8 months, that is the avg time for which Ola is able to retain the driver in the company, 1 being the lowest and 2 years being the highest.


```
In [ ]: df_final['Education'].value_counts(normalize = True)*100
```

Out[36]:

	proportion
Education	
2.0	33.683326
1.0	33.389332
0.0	32.927341

dtype: float64

- Almost 33% of the drivers are graduates.

Univariate Analysis

Categorical Variables

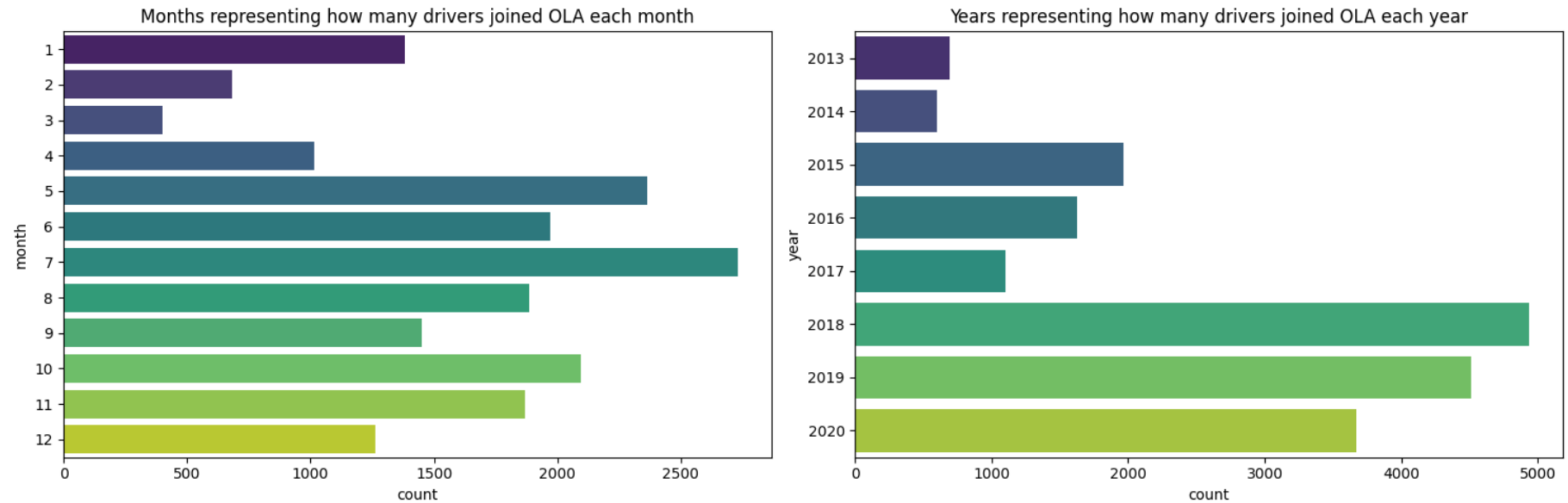
```
In [ ]: df['month'] = df['Dateofjoining'].dt.month
df['year'] = df['Dateofjoining'].dt.year

# Create subplots
fig, axes = plt.subplots(1, 2, figsize=(15, 5))

# Plot count of drivers joined each month
sns.countplot(y=df['month'], palette='viridis', ax=axes[0])
axes[0].set_title('Months representing how many drivers joined OLA each month')

# Plot count of drivers joined each year
sns.countplot(y=df['year'], palette='viridis', ax=axes[1])
axes[1].set_title('Years representing how many drivers joined OLA each year')

# Adjust layout
plt.tight_layout()
plt.show()
```



- July attracted the highest influx of drivers over an eight-year period.
- February and March had the lowest number of drivers joining OLA.
- The recruitment of drivers experienced a surge of approximately 500% post-2017.

```
In [ ]: # Convert lists to strings or leave as-is if not a list
df_final['Gender'] = df_final['Gender'].apply(lambda x: str(x) if isinstance(x, list) else x)
df_final['City'] = df_final['City'].apply(lambda x: str(x) if isinstance(x, list) else x)
```

```
In [ ]: df_final['Gender'] = df_final['Gender'].apply(lambda x: 1 if x >= 0.5 else 0)
df_final['Gender'].value_counts()
```

Out[39]:

	count
Gender	
0	1402
1	979

dtype: int64

```
In [ ]: df_final.head()
```

Out[40]:

	Driver_ID	Age	Gender	Education	Income	Joining_Designation	Grade	Total_Business_Value	Last_Quarterly_Rating	LastWorkingDate	Months
0	1.0	28.0	0	2.0	57387.0	1.0	1.0	1715580.0	2.0	NaN	3
1	2.0	31.0	0	2.0	67016.0	2.0	2.0	0.0	1.0	0.0	2
2	4.0	43.0	0	2.0	65603.0	2.0	2.0	350000.0	1.0	NaN	5
3	5.0	29.0	0	0.0	46368.0	1.0	1.0	120360.0	1.0	NaN	3
4	6.0	31.0	1	1.0	78728.0	3.0	3.0	1265000.0	2.0	0.0	5

```
In [ ]: plt.figure(figsize=(15, 15))

plt.subplot(421)
sns.countplot(data=df_final, x="Gender")

plt.subplot(422)
sns.countplot(data=df_final, x="City")
plt.xticks(rotation=45) # Corrected rotation parameter

plt.subplot(423)
sns.countplot(data=df_final, x="Joining_Designation")

plt.subplot(424)
sns.countplot(data=df_final, x="Education")

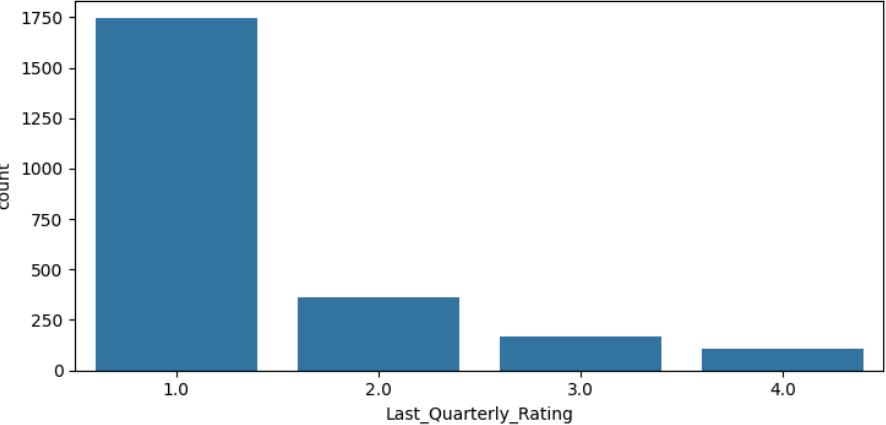
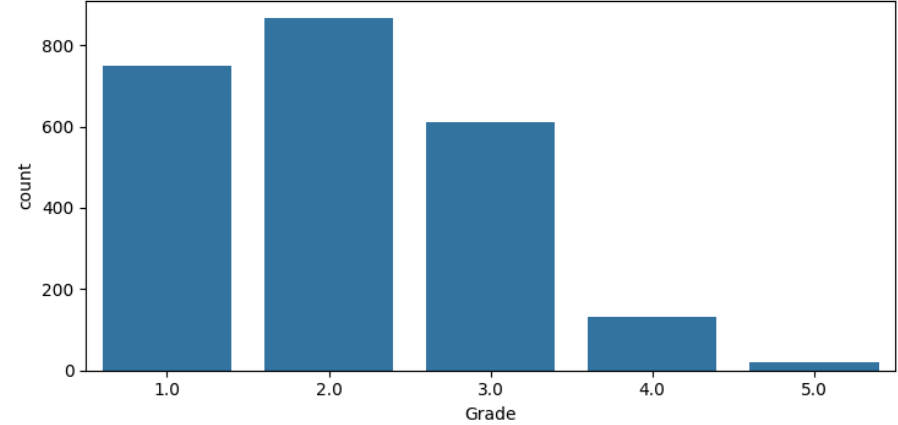
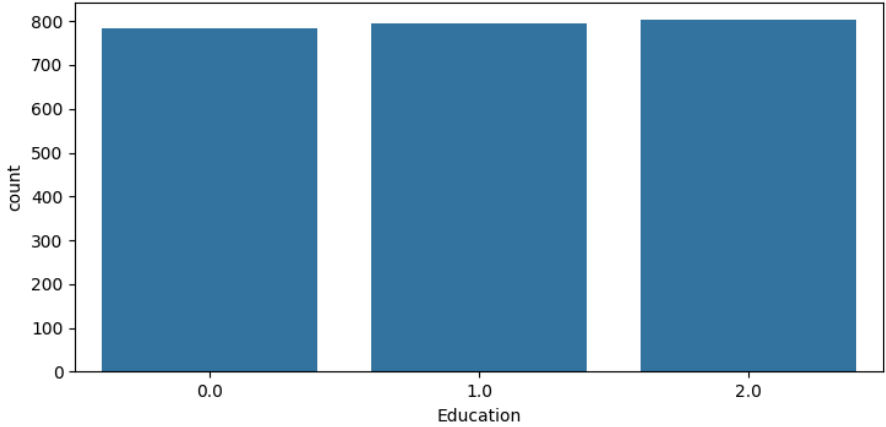
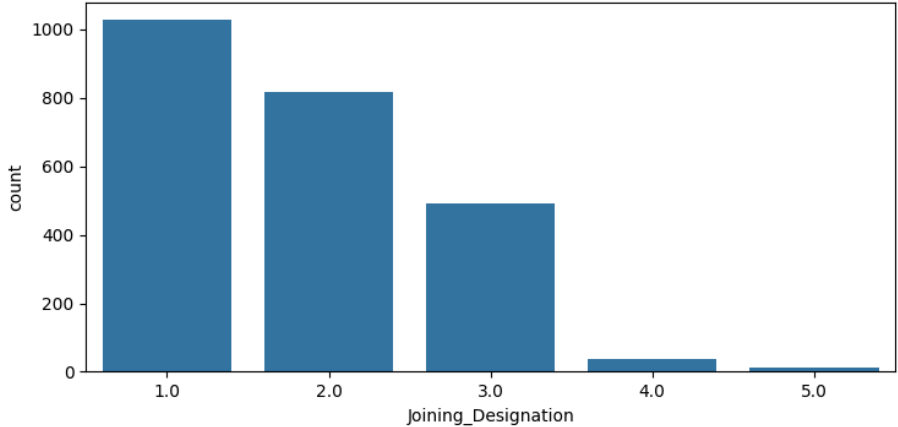
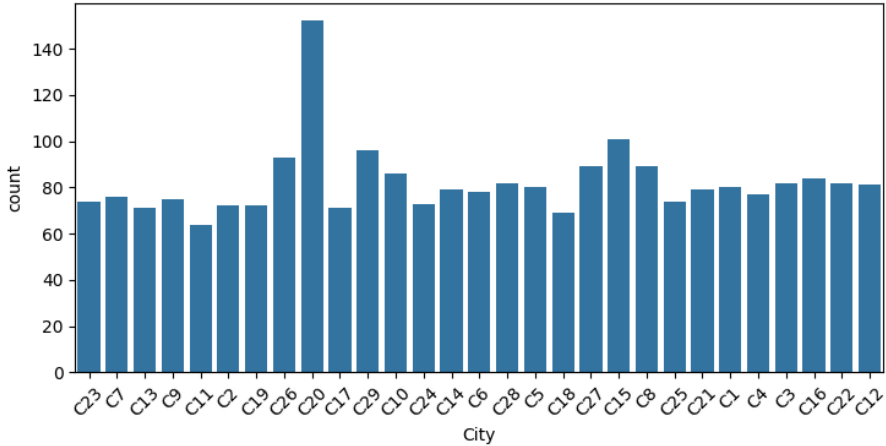
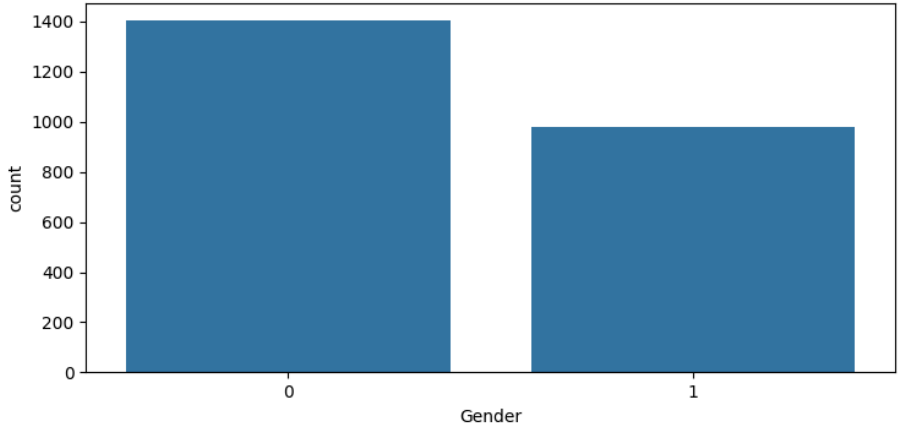
plt.subplot(425)
sns.countplot(data=df_final, x="Grade")

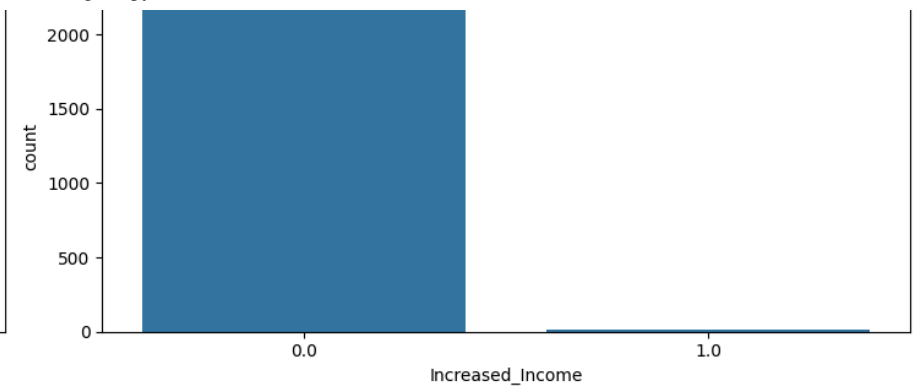
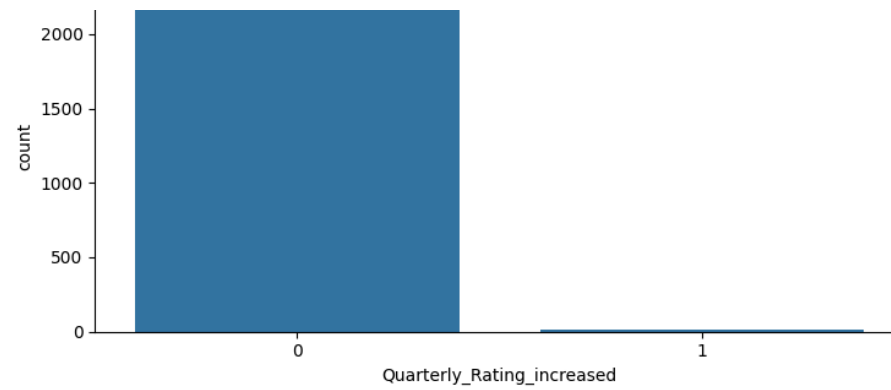
plt.subplot(426)
sns.countplot(data=df_final, x="Last_Quarterly_Rating")

plt.subplot(427)
sns.countplot(data=df_final, x="Quarterly_Rating_increased")

plt.subplot(428)
sns.countplot(data=df_final, x="Increased_Income")

plt.tight_layout()
plt.show()
```

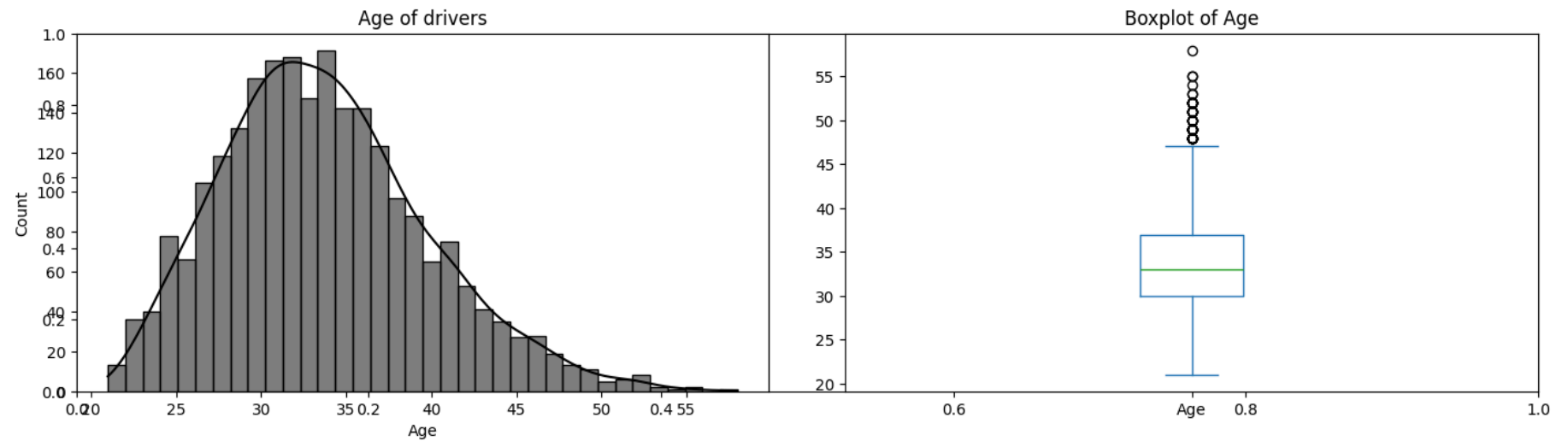





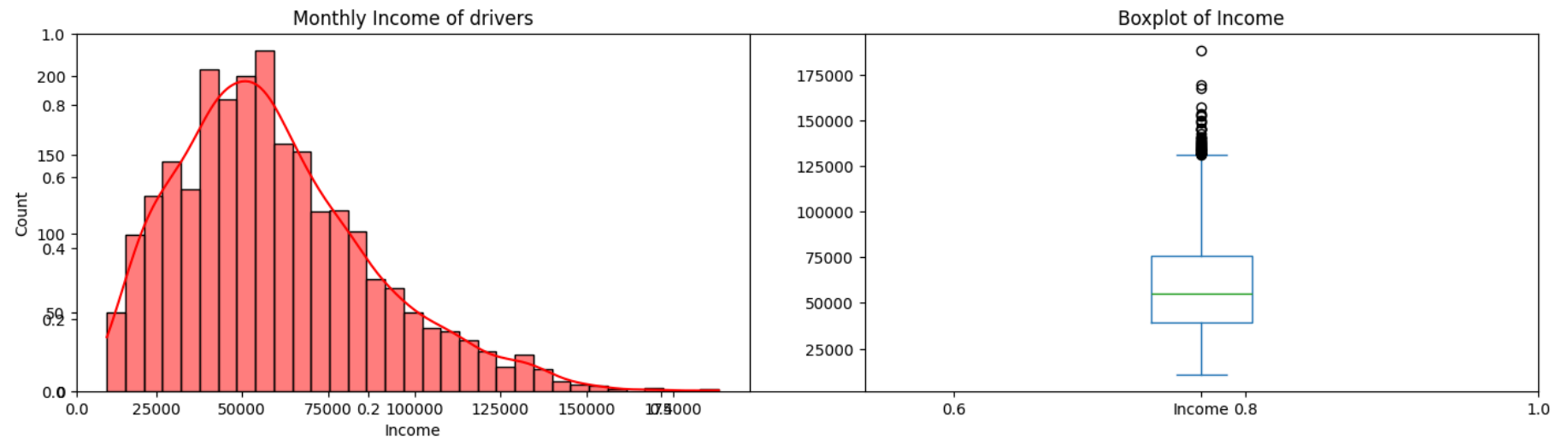
Continuous Variables

Histogram/ density plots and Box plots for categorical feat

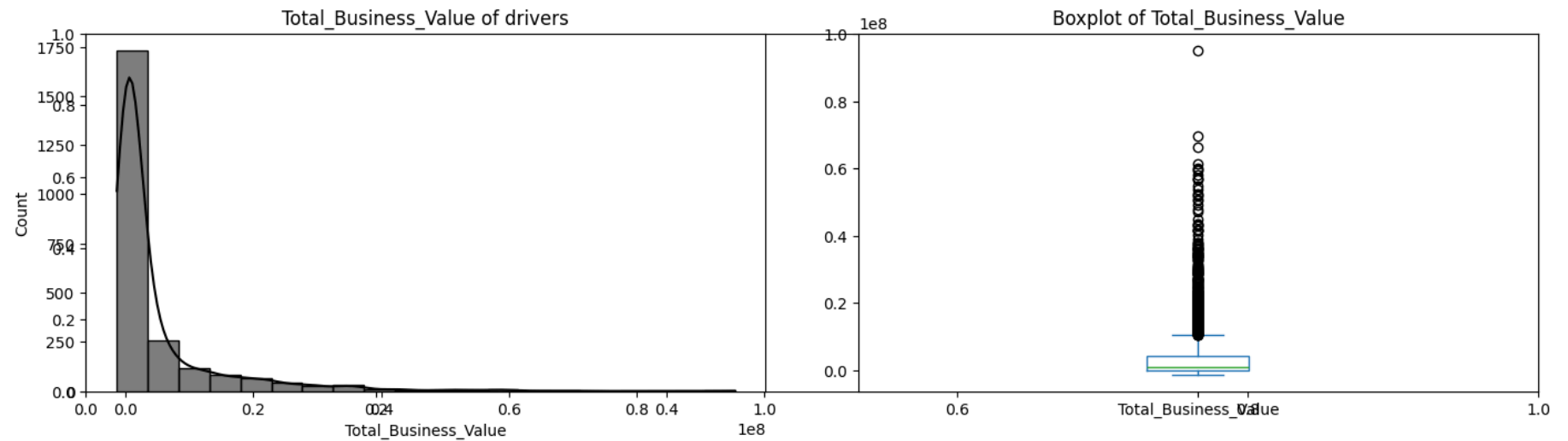

```
In [ ]: plt.subplots(figsize=(15,5))
plt.subplot(121)
sns.histplot(df_final['Age'],color='black', kde=True)
plt.title("Age of drivers")
plt.subplot(122)
df_final['Age'].plot.box(title='Boxplot of Age')
plt.tight_layout(pad=3)
```



```
In [ ]: plt.subplots(figsize=(15,5))
plt.subplot(121)
sns.histplot(df_final['Income'],color='red', kde=True)
plt.title("Monthly Income of drivers")
plt.subplot(122)
df_final['Income'].plot.box(title='Boxplot of Income')
plt.tight_layout(pad=3)
```



```
In [ ]: plt.subplots(figsize=(15,5))
plt.subplot(121)
sns.histplot(df_final['Total_Business_Value'],color='black', kde=True, bins=20)
plt.title("Total_Business_Value of drivers")
plt.subplot(122)
df_final['Total_Business_Value'].plot.box(title='Boxplot of Total_Business_Value')
plt.tight_layout(pad=3)
```

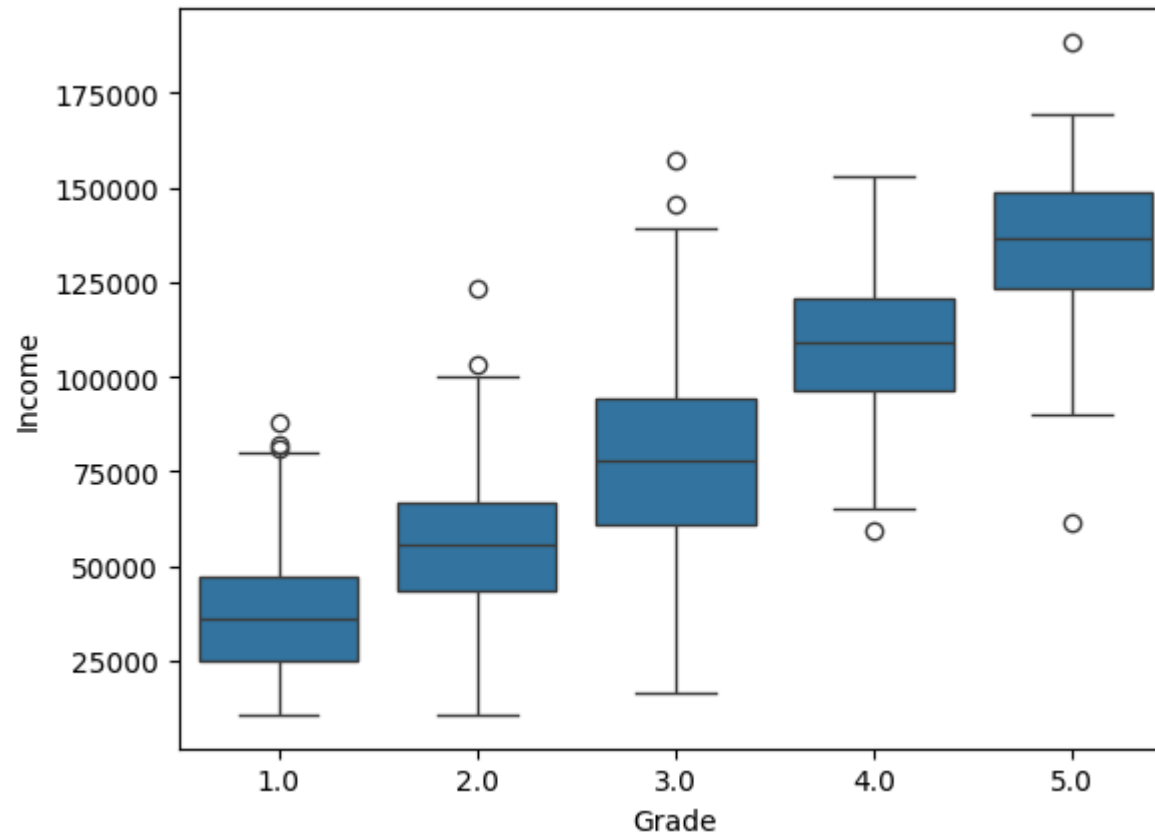


The right-skewed distribution of total business value suggests potential outliers within the data.

Bi-variate Analysis

```
In [ ]: sns.boxplot(data=df_final, x='Grade', y='Income')
```

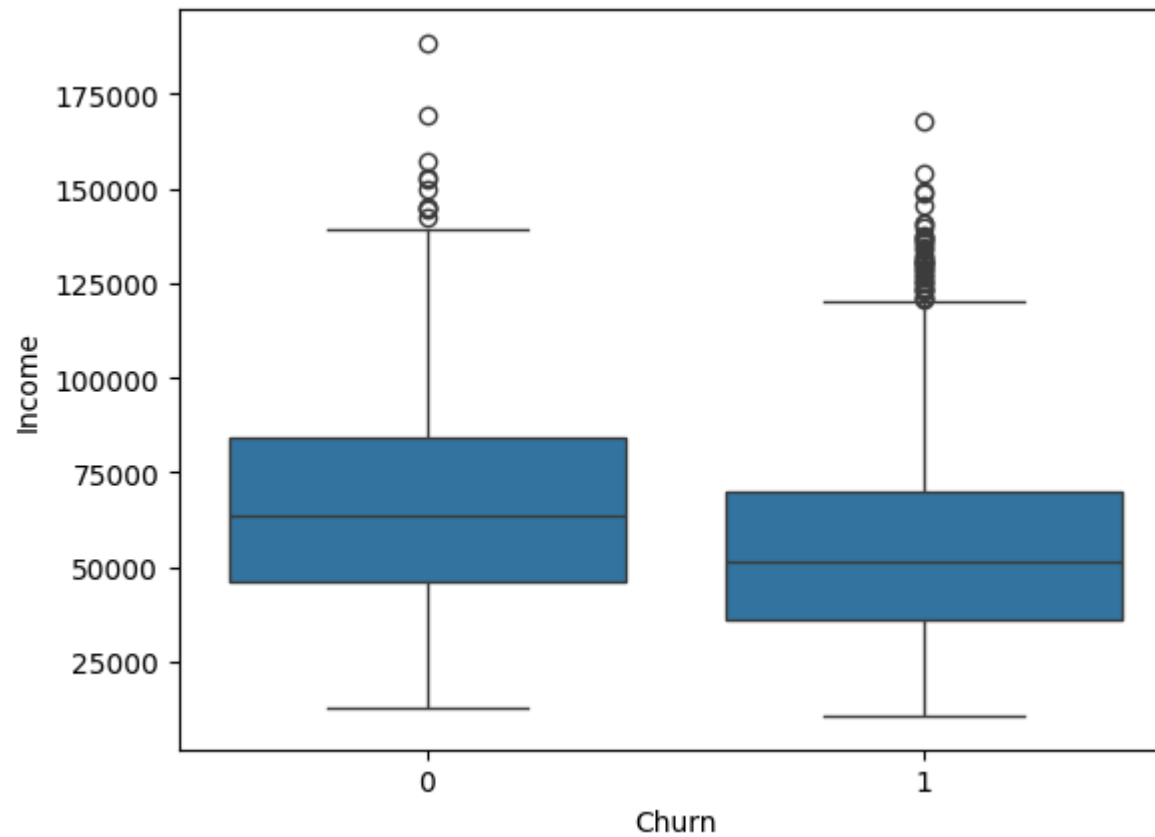
```
Out[45]: <Axes: xlabel='Grade', ylabel='Income'>
```



- Higher the Grade of the driver , higher the income of the driver.

```
In [ ]: sns.boxplot(data=df_final, x='Churn', y='Income')
```

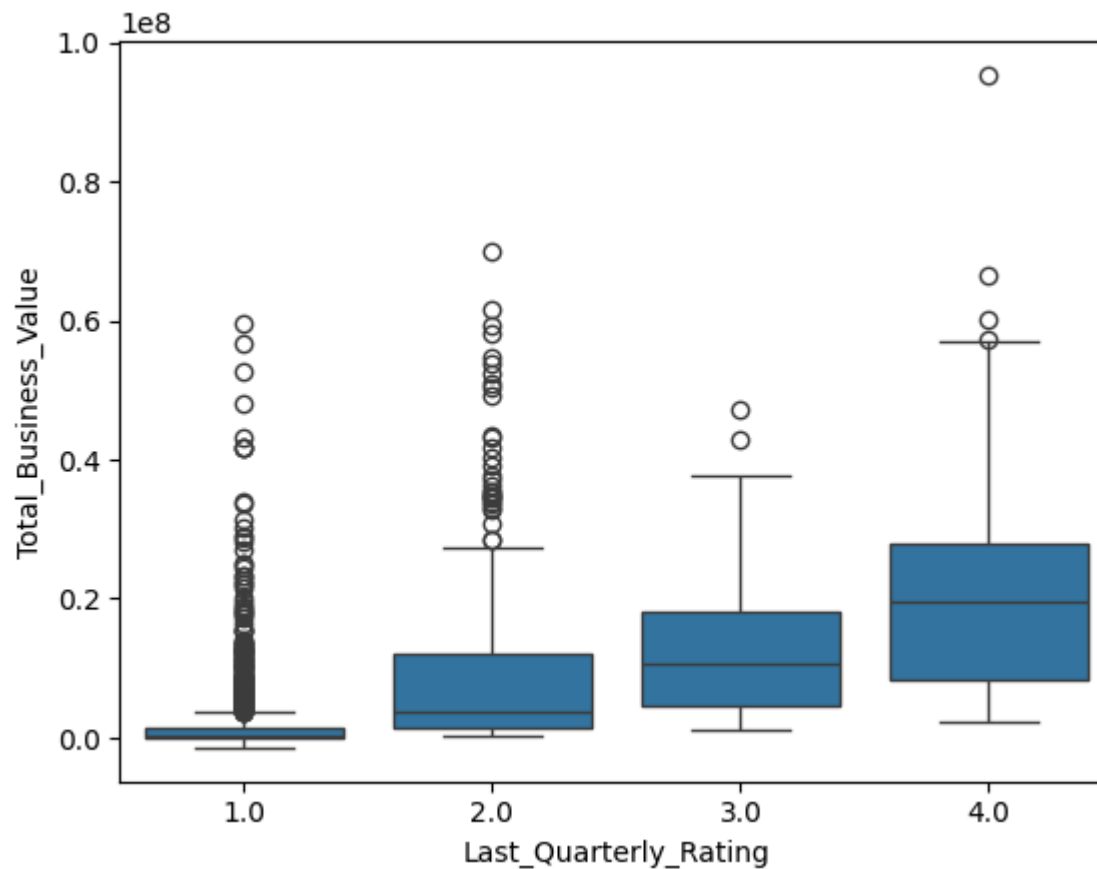
```
Out[46]: <Axes: xlabel='Churn', ylabel='Income'>
```



- People who left the company have lower income compared to who stayed.

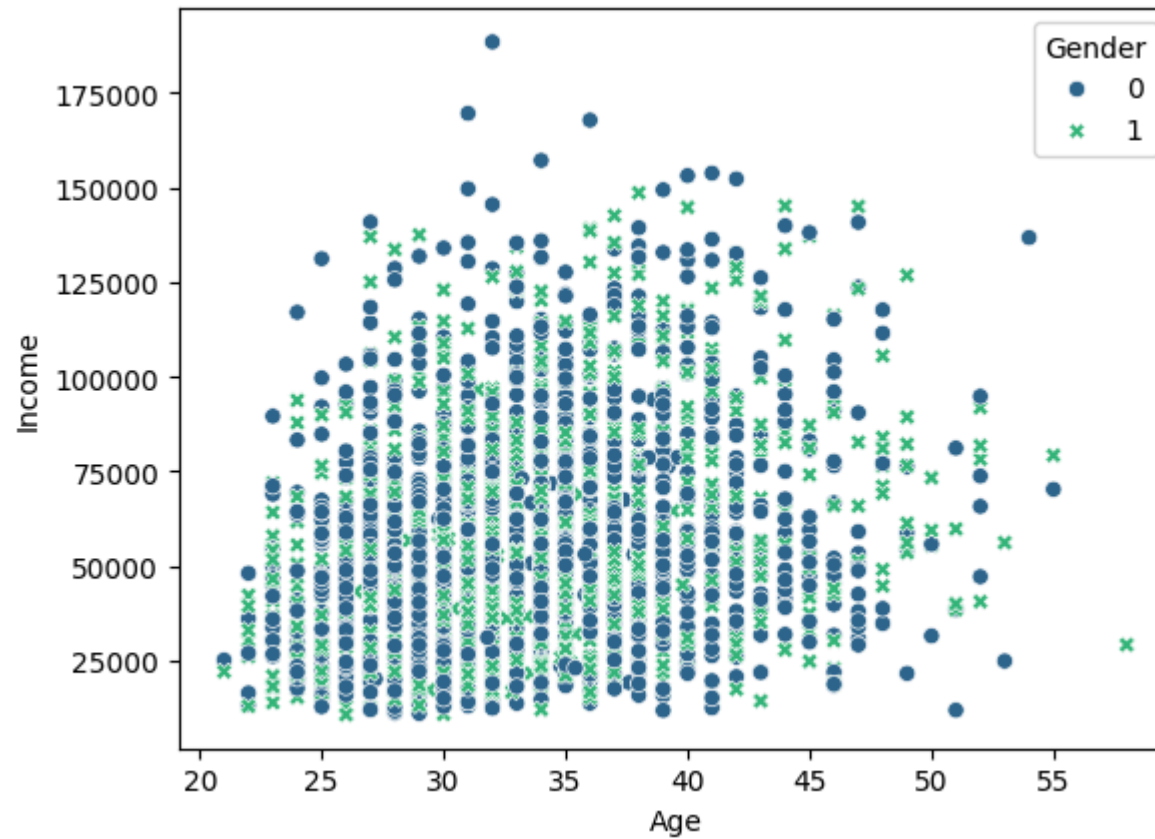
```
In [ ]: sns.boxplot(data=df_final, x='Last_Quarterly_Rating', y='Total_Business_Value')
```

```
Out[47]: <Axes: xlabel='Last_Quarterly_Rating', ylabel='Total_Business_Value'>
```



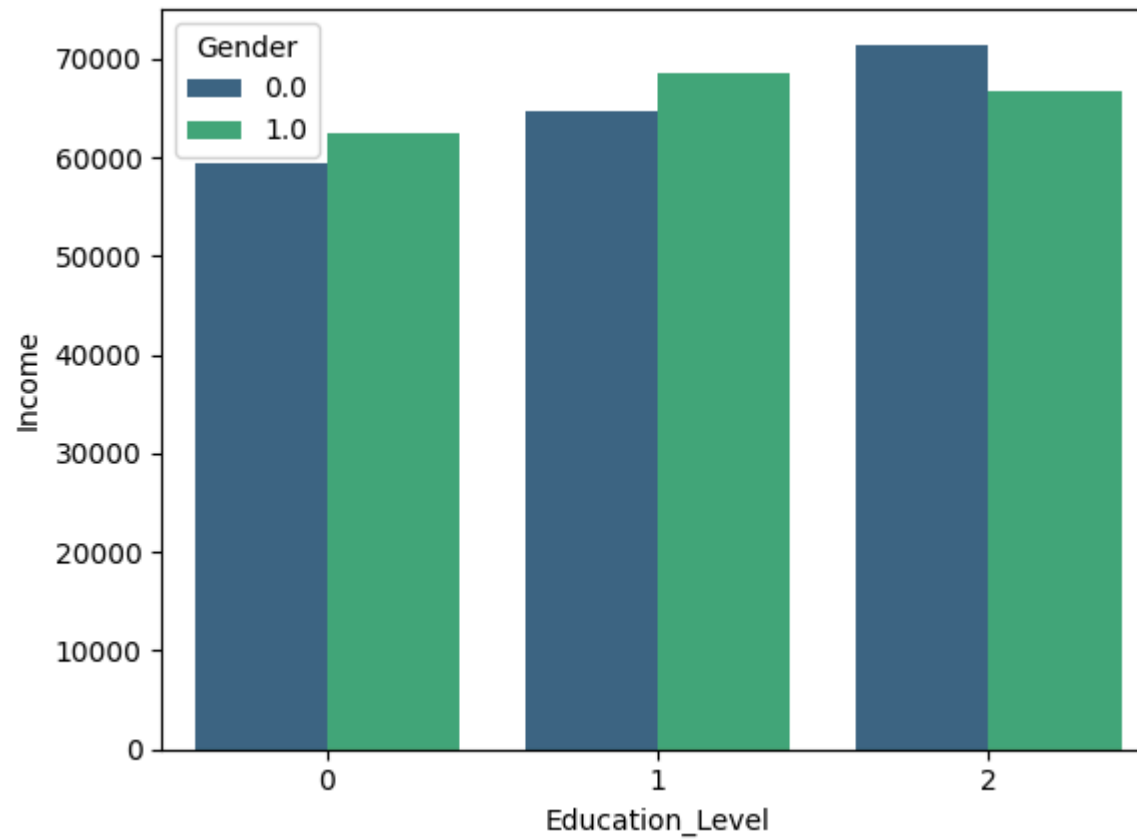
- Generally the driver with the higher rating earns more.
- There are a lot of outliers present with the rating and 1 and 2.

```
In [ ]: sns.scatterplot(data=df_final, x='Age', y='Income', hue='Gender', style='Gender', palette='viridis')  
plt.show()
```



```
In [ ]: sns.barplot(data=df, x='Education_Level', y='Income', hue='Gender', ci=None, palette='viridis')
```

```
Out[49]: <Axes: xlabel='Education_Level', ylabel='Income'>
```



- Females with 10th and 12th pass earn more than males.
- Graduates male earns more than female driver.


```
In [ ]: city_mapping = {
        'C1': 1, 'C2': 2, 'C3': 3, 'C4': 4, 'C5': 5, 'C6': 6, 'C7': 7, 'C8': 8, 'C9': 9, 'C10': 10,
        'C11': 11, 'C12': 12, 'C13': 13, 'C14': 14, 'C15': 15, 'C16': 16, 'C17': 17, 'C18': 18, 'C19': 19, 'C20': 20,
        'C21': 21, 'C22': 22, 'C23': 23, 'C24': 24, 'C25': 25, 'C26': 26, 'C27': 27, 'C28': 28, 'C29': 29
    }

    # Map the cities using the dictionary
    df_final['City'] = df_final['City'].map(city_mapping)
    df_final['City'].value_counts().head()
```

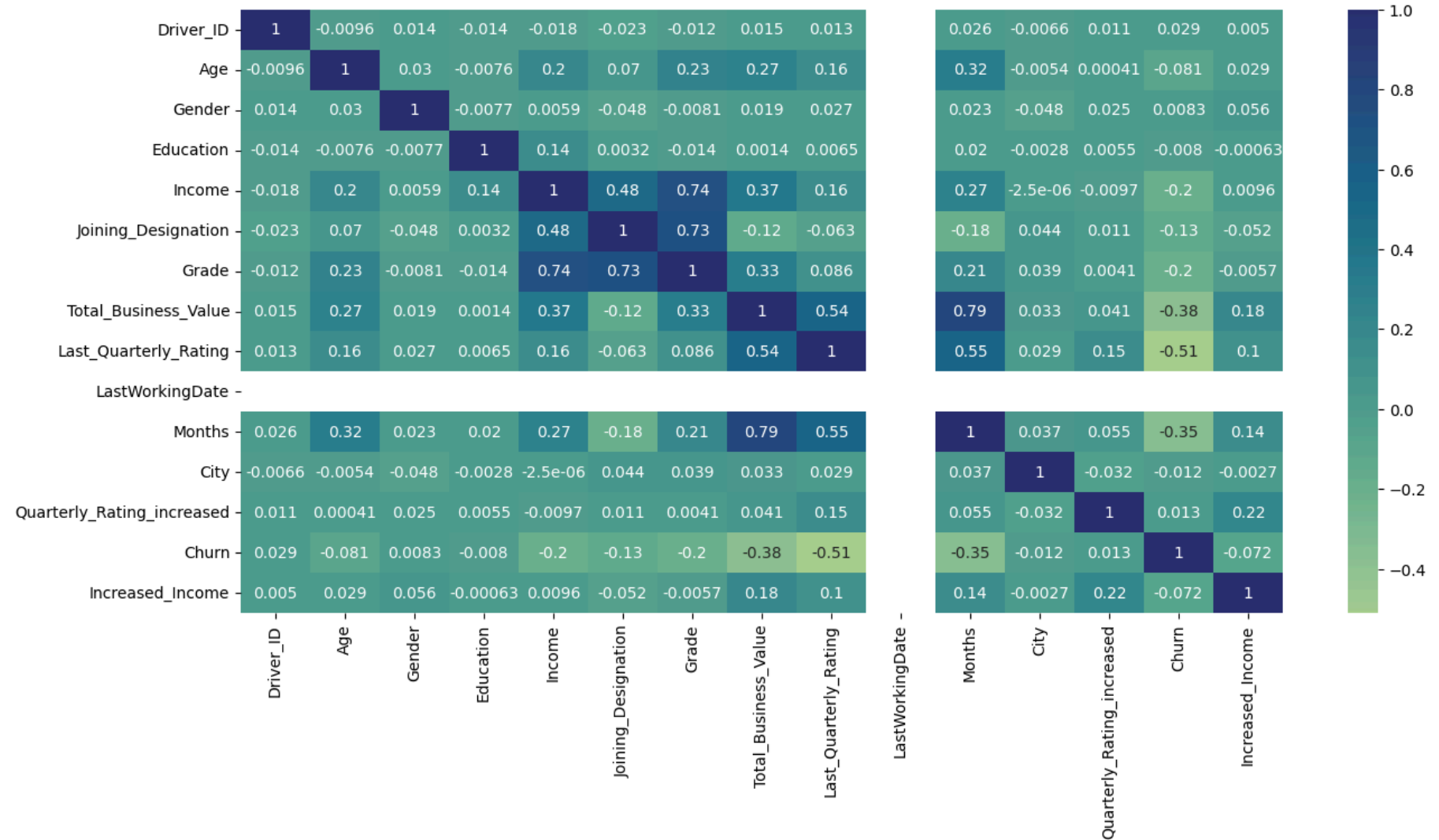
Out[50]:

	count
City	
20	152
15	101
29	96
26	93
27	89

dtype: int64

```
In [ ]: plt.figure(figsize=(15, 7))

sns.heatmap(df_final.corr(method="pearson", numeric_only = True), annot=True, cmap="crest")
plt.show()
```



Pre-processing and Model buidling

Standardization

```
In [ ]: df_final.columns
```

```
Out[52]: Index(['Driver_ID', 'Age', 'Gender', 'Education', 'Income',  
              'Joining_Designation', 'Grade', 'Total_Business_Value',  
              'Last_Quarterly_Rating', 'LastWorkingDate', 'Months', 'City',  
              'Dateofjoining', 'Quarterly_Rating_increased', 'Churn',  
              'Increased_Income'],  
           dtype='object')
```

```
In [ ]: X = df_final.drop(["Churn", 'Dateofjoining', 'LastWorkingDate'], axis = 1)  
y = df_final["Churn"]  
X_cols = X.columns
```

Train & Test Split

```
In [ ]: from sklearn.model_selection import train_test_split  
  
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, shuffle=True)  
print("X_train Shape: ", X_train.shape)  
print("X_test Shape: ", X_test.shape)  
print("y_train Shape: ", y_train.shape)  
print("y_test Shape: ", y_test.shape)  
  
X_train Shape: (1904, 13)  
X_test Shape: (477, 13)  
y_train Shape: (1904,)  
y_test Shape: (477,)
```

```
In [ ]: from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

Random Forest Classifier with Class Imbalance

```
In [ ]: from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

rfc = RandomForestClassifier(n_estimators=100, min_samples_leaf=3, criterion='entropy', n_jobs=-1, random_state=42)

# Train the classifier
rfc.fit(X_train, y_train)

# Predict on the train and validation set
pred_train = rfc.predict(X_train)
pred_test = rfc.predict(X_test)

print(f'Train Accuracy: {accuracy_score(pred_train, y_train)}')
print(f'Validation Accuracy: {accuracy_score(pred_test, y_test)}')
```

Train Accuracy: 0.9275210084033614
Validation Accuracy: 0.8218029350104822

```
In [ ]: rfc.feature_importances_
```

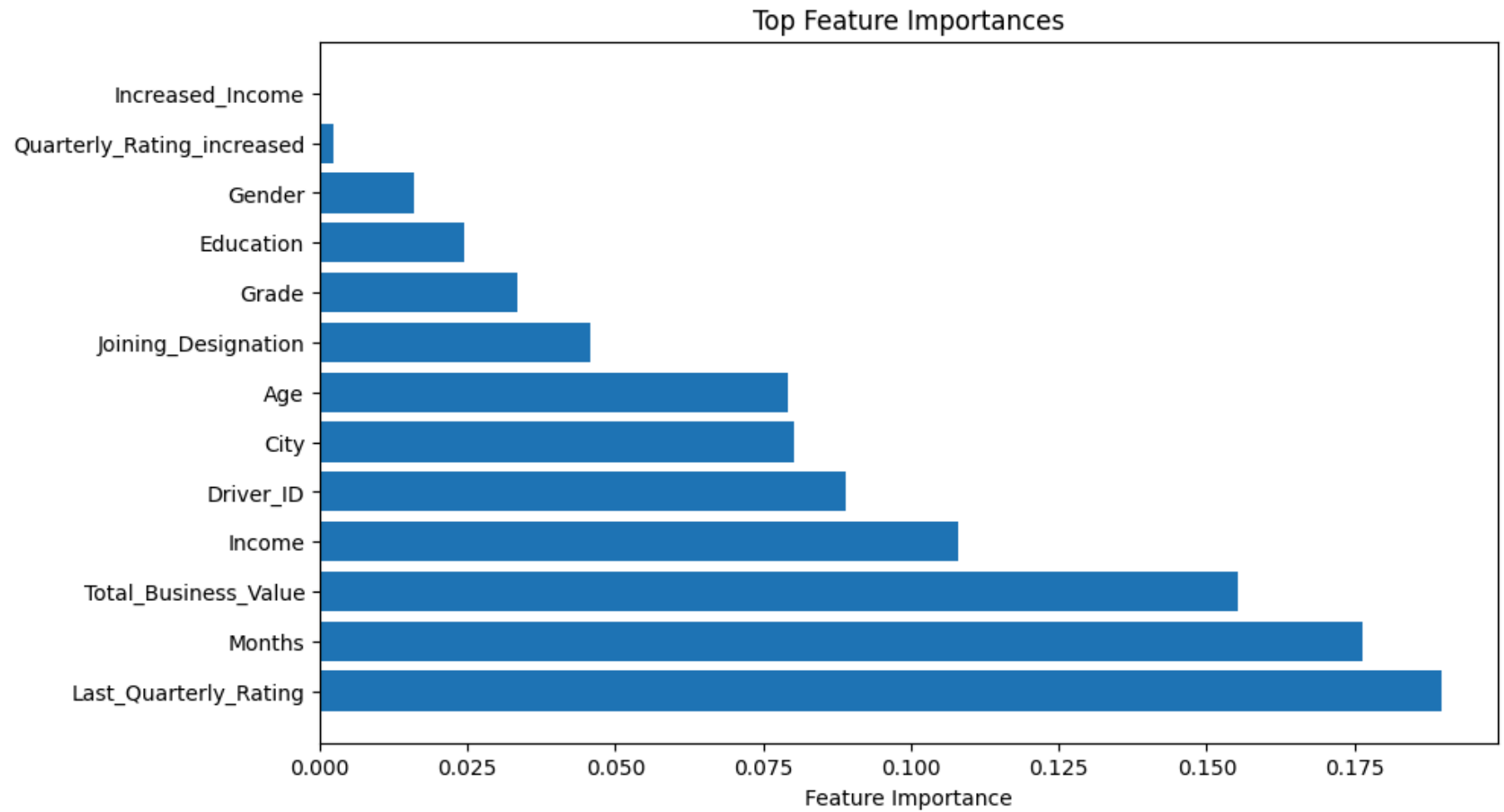
```
Out[57]: array([0.08901216, 0.07923765, 0.01597313, 0.02454053, 0.10796782,
                0.04569428, 0.03344125, 0.15525995, 0.18976513, 0.17638038,
                0.08014471, 0.00219994, 0.00038307])
```

```
In [ ]: # Get feature importances
feature_importances = rfc.feature_importances_

# Get the features and their importance scores
features = X_cols
importances = feature_importances

# Sort the importances
sorted_idx = importances.argsort()

# Plot the most important features
plt.figure(figsize=(10, 6))
plt.barh(range(len(importances[sorted_idx])), importances[sorted_idx], align='center')
plt.yticks(range(len(importances[sorted_idx])), [features[i] for i in sorted_idx])
plt.xlabel('Feature Importance')
plt.title('Top Feature Importances')
plt.gca().invert_yaxis() # To have the most important features at the top
plt.show()
```



Balancing Dataset using SMOTE

```
In [ ]: !pip install imblearn
```

```
Collecting imblearn
  Downloading imblearn-0.0-py2.py3-none-any.whl.metadata (355 bytes)
Collecting imbalanced-learn (from imblearn)
  Downloading imbalanced_learn-0.13.0-py3-none-any.whl.metadata (8.8 kB)
Requirement already satisfied: numpy<3,>=1.24.3 in /usr/local/lib/python3.10/dist-packages (from imbalanced-learn->imblearn) (1.26.4)
Requirement already satisfied: scipy<2,>=1.10.1 in /usr/local/lib/python3.10/dist-packages (from imbalanced-learn->imblearn) (1.13.1)
Requirement already satisfied: scikit-learn<2,>=1.3.2 in /usr/local/lib/python3.10/dist-packages (from imbalanced-learn->imblearn) (1.6.0)
Collecting sklearn-compat<1,>=0.1 (from imbalanced-learn->imblearn)
  Downloading sklearn_compat-0.1.3-py3-none-any.whl.metadata (18 kB)
Requirement already satisfied: joblib<2,>=1.1.1 in /usr/local/lib/python3.10/dist-packages (from imbalanced-learn->imblearn) (1.4.2)
Requirement already satisfied: threadpoolctl<4,>=2.0.0 in /usr/local/lib/python3.10/dist-packages (from imbalanced-learn->imblearn) (3.5.0)
Downloading imblearn-0.0-py2.py3-none-any.whl (1.9 kB)
Downloading imbalanced_learn-0.13.0-py3-none-any.whl (238 kB)
238.4/238.4 kB 6.5 MB/s eta 0:00:00
Downloading sklearn_compat-0.1.3-py3-none-any.whl (18 kB)
Installing collected packages: sklearn-compat, imbalanced-learn, imblearn
Successfully installed imbalanced-learn-0.13.0 imblearn-0.0 sklearn-compat-0.1.3
```

```
In [ ]: from imblearn.over_sampling import SMOTE

print("Before OverSampling, counts of label '1': {}".format(sum(y_train == 1)))
print("Before OverSampling, counts of label '0': {} \n".format(sum(y_train == 0)))

smt = SMOTE(random_state = 42)
X_train, y_train = smt.fit_resample(X_train, y_train.ravel())

print('After OverSampling, the shape of train_X: {}'.format(X_train.shape))
print('After OverSampling, the shape of train_y: {} \n'.format(y_train.shape))

print("After OverSampling, counts of label '1': {}".format(sum(y_train == 1)))
print("After OverSampling, counts of label '0': {}".format(sum(y_train == 0)))
```

Before OverSampling, counts of label '1': 1289

Before OverSampling, counts of label '0': 615

After OverSampling, the shape of train_X: (2578, 13)

After OverSampling, the shape of train_y: (2578,)

After OverSampling, counts of label '1': 1289

After OverSampling, counts of label '0': 1289

Random Forest (Ensemble Bagging) after SMOTE

```
In [ ]: from sklearn.model_selection import GridSearchCV
from sklearn.metrics import classification_report, confusion_matrix, ConfusionMatrixDisplay

# Define parameters for grid search
params = {
    'max_depth': [2, 3, 4],
    'n_estimators': [50, 100, 150, 200]
}

# Initialize RandomForestClassifier with balanced subsample
rfc1 = RandomForestClassifier(class_weight="balanced_subsample")

# Perform grid search
c = GridSearchCV(estimator=rfc1, param_grid=params, n_jobs=-1, cv=3, verbose=True, scoring='f1')
c.fit(X_train, y_train)

# Print best parameters and score
print("Best parameters:", c.best_params_)
print("Best Score:", c.best_score_)

# Predict and print classification report
y_pred = c.predict(X_test)
print(classification_report(y_test, y_pred))

# Plot confusion matrix
cm = confusion_matrix(y_test, y_pred)
ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=c.classes_).plot()
```

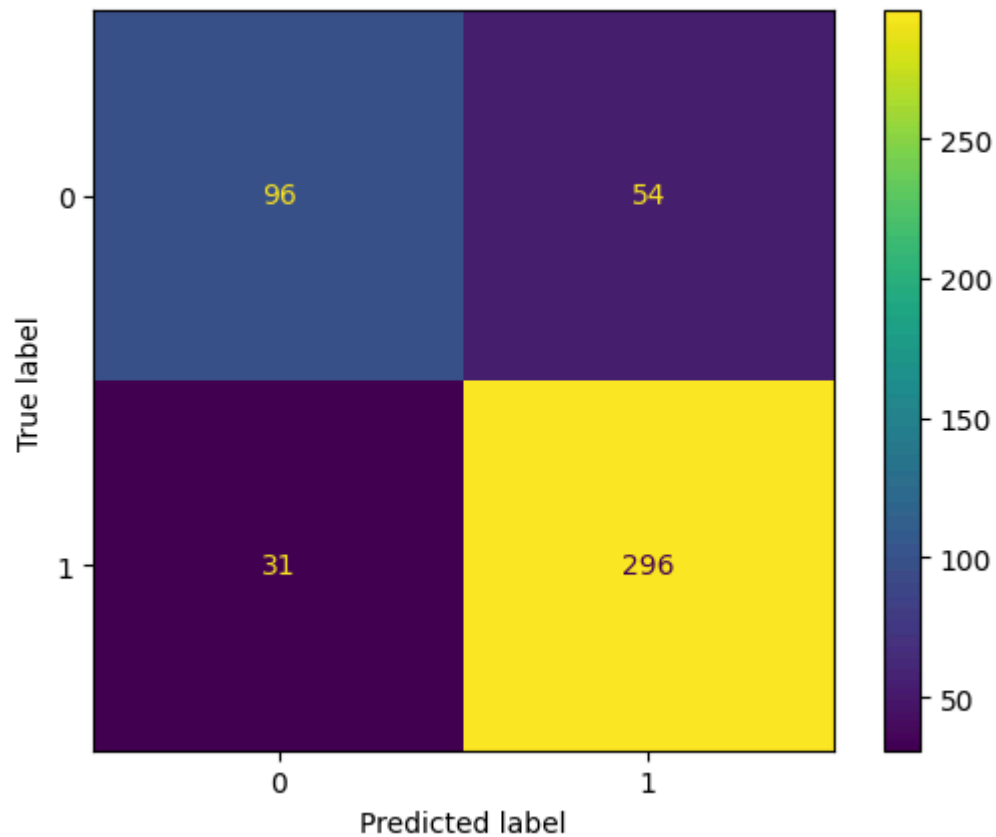
Fitting 3 folds for each of 12 candidates, totalling 36 fits

Best parameters: {'max_depth': 4, 'n_estimators': 100}

Best Score: 0.80985581338689

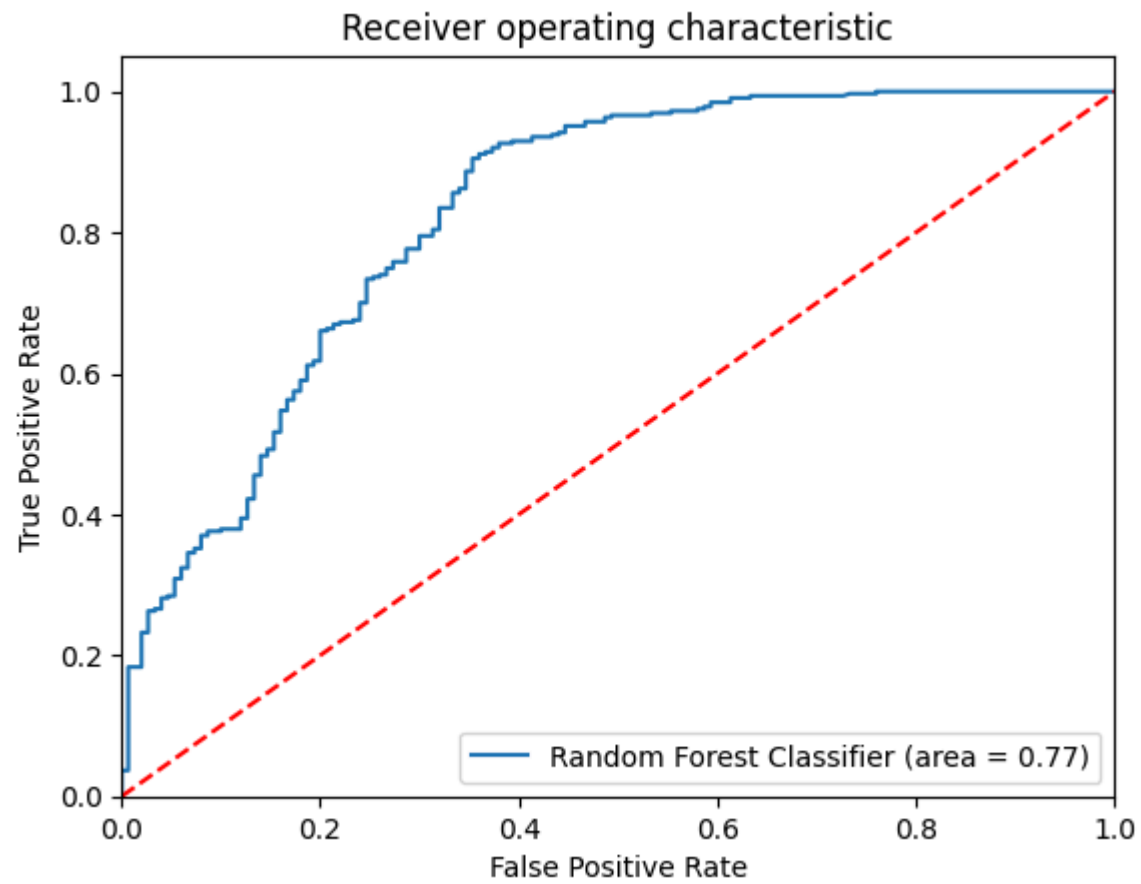
	precision	recall	f1-score	support
0	0.76	0.64	0.69	150
1	0.85	0.91	0.87	327
accuracy			0.82	477
macro avg	0.80	0.77	0.78	477
weighted avg	0.82	0.82	0.82	477

Out[62]: <sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x7dfd4da210c0>



```
In [ ]: from sklearn.metrics import roc_auc_score, roc_curve

logit_roc_auc=roc_auc_score(y_test,y_pred)
fpr,tpr,thresholds = roc_curve(y_test,c.predict_proba(X_test)[:,-1])
plt.figure()
plt.plot(fpr,tpr,label = 'Random Forest Classifier (area = %0.2f)' % logit_roc_auc)
plt.plot([0,1],[0,1], 'r--')
plt.xlim([0.0,1.0])
plt.ylim([0.0,1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver operating characteristic')
plt.legend(loc="lower right")
plt.show()
```



Ensemble Learning: Boosting

```
In [ ]: from sklearn.ensemble import GradientBoostingClassifier
params = {
    "max_depth": [2, 3, 4],
    "loss": ["log_loss", "exponential"],
    "subsample": [0.1, 0.2, 0.5, 0.8, 1],
    "learning_rate": [0.1, 0.2, 0.3],
    "n_estimators": [50, 100, 150, 200]
}

gbc = GradientBoostingClassifier()
c = GridSearchCV(estimator=gbc, param_grid=params, cv=3, n_jobs=-1, verbose=True)

c.fit(X_train, y_train)
print("Best Params: ", c.best_params_)
print("Best Score: ", c.best_score_)

y_pred = c.predict(X_test)

print(classification_report(y_test, y_pred))

cm = confusion_matrix(y_test, y_pred)

ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=c.classes_).plot()
```

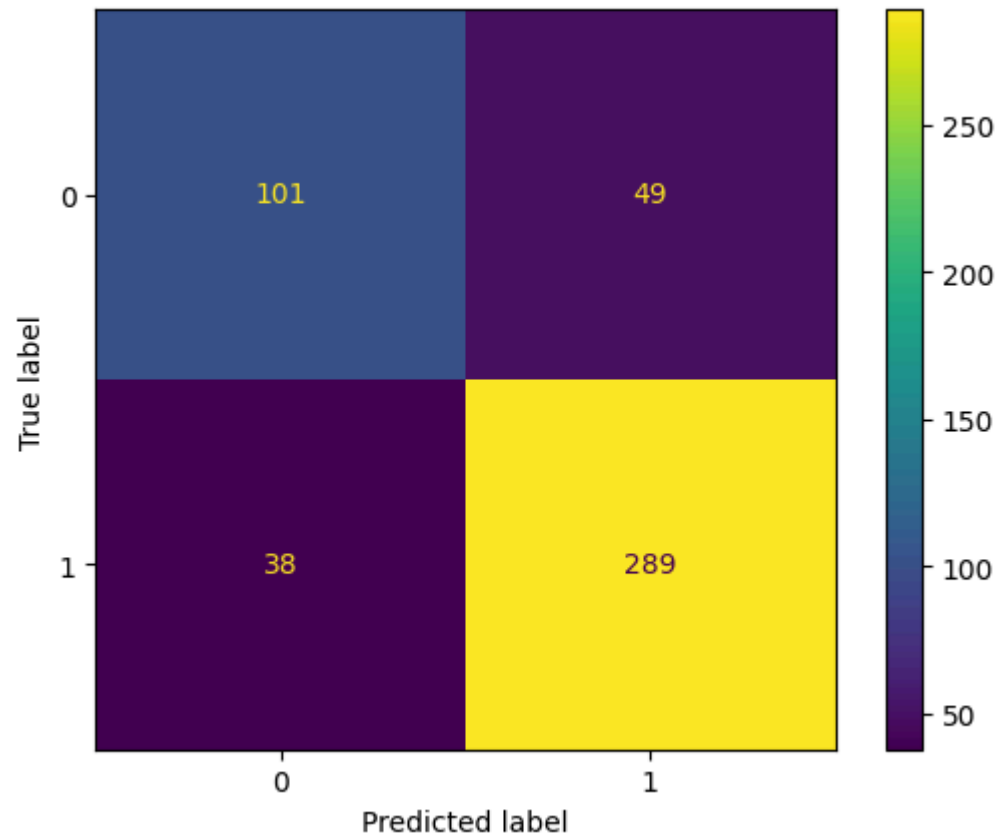
Fitting 3 folds for each of 360 candidates, totalling 1080 fits

Best Params: {'learning_rate': 0.1, 'loss': 'log_loss', 'max_depth': 4, 'n_estimators': 150, 'subsample': 0.5}

Best Score: 0.8429127072222071

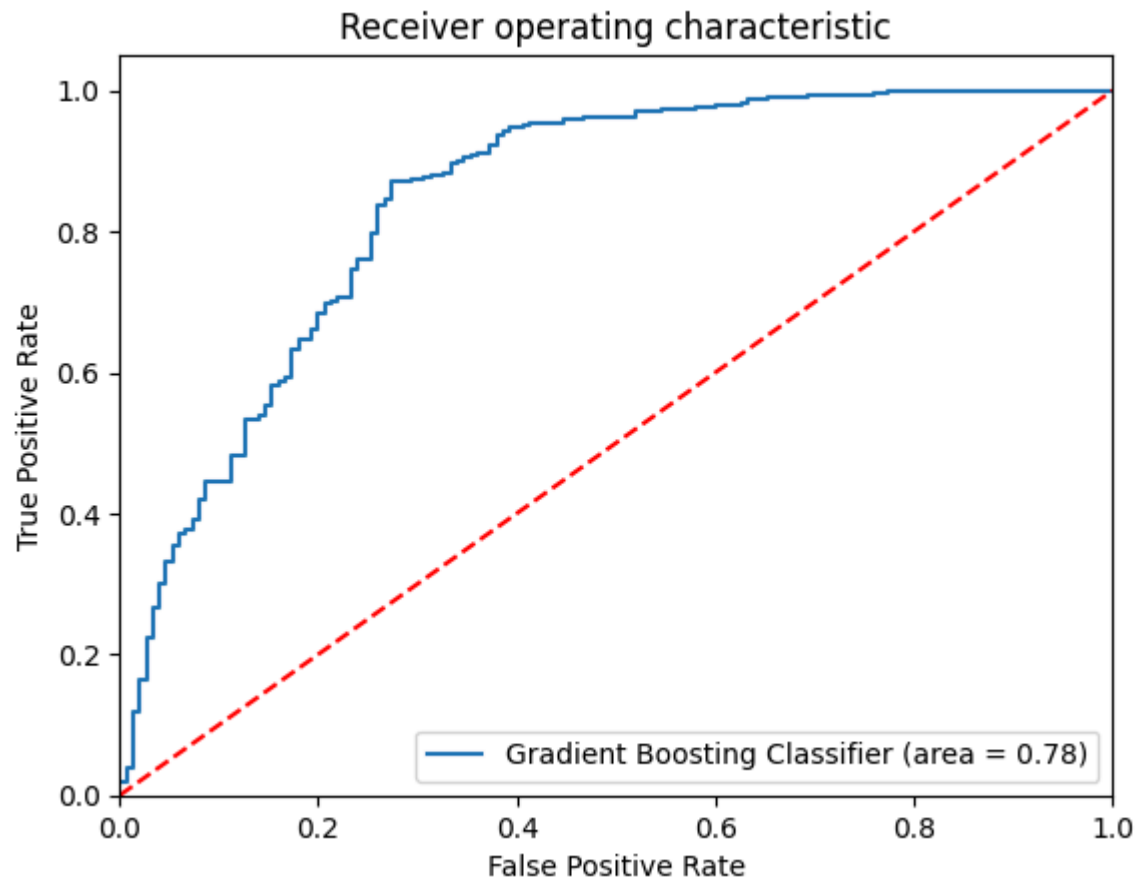
	precision	recall	f1-score	support
0	0.73	0.67	0.70	150
1	0.86	0.88	0.87	327
accuracy			0.82	477
macro avg	0.79	0.78	0.78	477
weighted avg	0.81	0.82	0.82	477

Out[64]: <sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x7dfd4d97bcd0>



```
In [ ]: logit_roc_auc = roc_auc_score(y_test,y_pred)
fpr, tpr, thresholds = roc_curve(y_test, c.predict_proba(X_test)[:,1])

plt.figure()
plt.plot(fpr, tpr, label='Gradient Boosting Classifier (area = %0.2f)' % logit_roc_auc)
plt.plot([0,1],[0,1], 'r--')
plt.xlim([0.0,1.0])
plt.ylim([0.0,1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver operating characteristic')
plt.legend(loc="lower right")
plt.show()
```



XGBoost Classifier

```
In [ ]: !pip install xgboost
```

```
Collecting xgboost
  Downloading xgboost-2.1.3-py3-none-manylinux_2_28_x86_64.whl.metadata (2.1 kB)
Requirement already satisfied: numpy in /usr/local/lib/python3.10/dist-packages (from xgboost) (1.26.4)
Collecting nvidia-nccl-cu12 (from xgboost)
  Downloading nvidia_nccl_cu12-2.23.4-py3-none-manylinux2014_x86_64.whl.metadata (1.8 kB)
Requirement already satisfied: scipy in /usr/local/lib/python3.10/dist-packages (from xgboost) (1.13.1)
Downloading xgboost-2.1.3-py3-none-manylinux_2_28_x86_64.whl (153.9 MB)
  ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 153.9/153.9 MB 6.9 MB/s eta 0:00:00
Downloading nvidia_nccl_cu12-2.23.4-py3-none-manylinux2014_x86_64.whl (199.0 MB)
  ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 199.0/199.0 MB 5.3 MB/s eta 0:00:00
Installing collected packages: nvidia-nccl-cu12, xgboost
Successfully installed nvidia-nccl-cu12-2.23.4 xgboost-2.1.3
```

```
In [ ]: import xgboost as xgb
from xgboost import XGBClassifier

model = xgb.XGBClassifier(class_weight = "balanced")

model.fit(X_train, y_train)

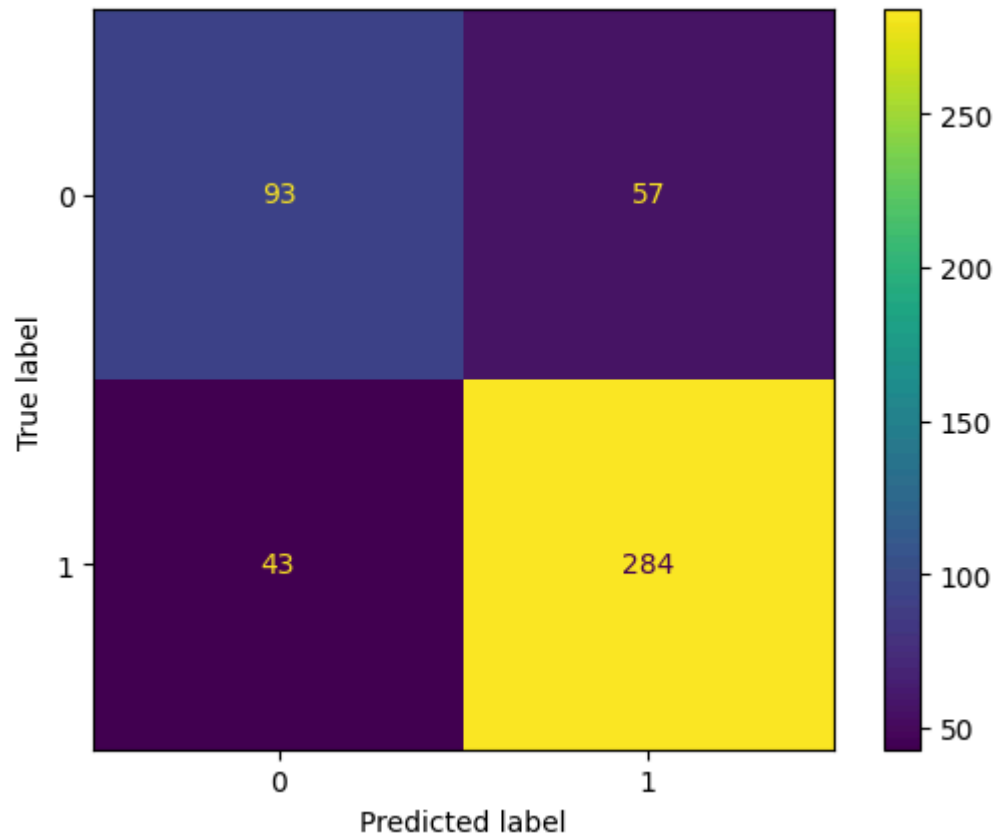
y_pred = model.predict(X_test)
print("XGBoost Classifier Score: ", model.score(X_test, y_test))
print("\n", classification_report(y_test, y_pred))
```

XGBoost Classifier Score: 0.790356394129979

	precision	recall	f1-score	support
0	0.68	0.62	0.65	150
1	0.83	0.87	0.85	327
accuracy			0.79	477
macro avg	0.76	0.74	0.75	477
weighted avg	0.79	0.79	0.79	477

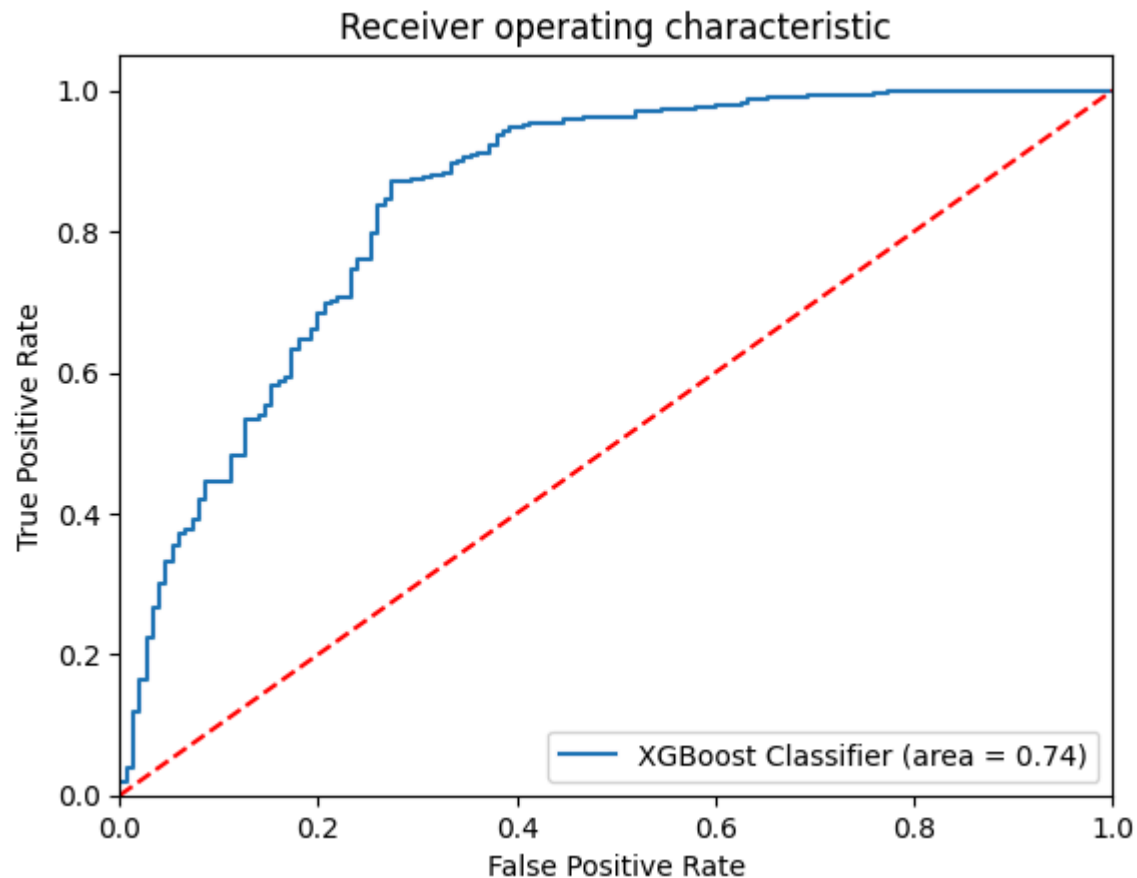

```
In [ ]: cm = confusion_matrix(y_test, y_pred)  
ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=model.classes_).plot()
```

Out[68]: <sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x7dfd556fd090>



```
In [ ]: logit_roc_auc=roc_auc_score(y_test,y_pred)
fpr, tpr, thresholds=roc_curve(y_test, c.predict_proba(X_test)[: ,1])

plt.figure()
plt.plot(fpr, tpr, label='XGBoost Classifier (area = %0.2f)' % logit_roc_auc)
plt.plot([0,1],[0,1], 'r--')
plt.xlim([0.0,1.0])
plt.ylim([0.0,1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver operating characteristic')
plt.legend(loc="lower right")
plt.show()
```



We find that the Gradient Boosting with SMOTE yields superior performance compared to other models, displaying higher recall and precision scores.

For the Random Forest model (After SMOTE):

- Precision for predicting class 0 is 76%, and for class 1, it is 85%.
- Recall for actual class 0 instances is 64%, and for class 1, it is 91%.
- f1 score for actual class 0 instances is 69%, and for class 1, it is 87%.
- The area under the ROC-AUC curve is 0.77.

Regarding the Gradient Boosting Classifier:

- Precision for predicting class 0 is 73%, and for class 1, it is 86%.
- Recall for actual class 0 instances is 67%, and for class 1, it is 88%.
- f1 score for actual class 0 instances is 70%, and for class 1, it is 87%.
- The area under the ROC-AUC curve is 0.78.

Regarding the XGBoost Classifier:

- Precision for predicting class 0 is 68%, and for class 1, it is 83%.
- Recall for actual class 0 instances is 62%, and for class 1, it is 87%.
- f1 score for actual class 0 instances is 65%, and for class 1, it is 85%.
- The area under the ROC-AUC curve is 0.74.

Actionable Insights and Recommendation

- There are 57% male employees and 43% female employees.
- Precision dropped after treatment of data imbalance and is performing better in Random Forest.
- The majority of the employees seem to be associated with city C20.
- Employees whose monthly salary has not increased are more likely to leave the organization. The company should engage with these drivers to help them earn more through bonuses and perks.
- The most important features are Last_Quarterly_Rating, Total_Business_Value, and Quarterly_Rating_Increased. The company should track these features as predictors.
- Employees whose quarterly rating has increased are less likely to leave the organization. Implementing a reward system for customers who provide feedback and rate drivers could further encourage driver retention.

- We observe that the recall score for target 0 is not very high, possibly due to the small, unbalanced dataset. More data could address this issue.